

claude-windows / 068560d3-1ca2-4387-981a-d2d6bca55414

Metadata

```
- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\068560d3-1ca2-4387-981a-d2d6bca55414\subagents\workflows\wf_b3a30c42-346\agent-a8516cfbdeb4b9016.jsonl`
- SHA-256: `2700dd7ba757c467f7eab0c69c867631df2ae3d488bbec9567c8b2ad5d58e5e3`
- Source modified: `2026-06-16T02:27:45+00:00`
- Imported at: `2026-07-05T16:48:28+00:00`
- project: `wf_b3a30c42-346`
- session_id: `068560d3-1ca2-4387-981a-d2d6bca55414`
```

Transcript

1. user (2026-06-16T02:24:52.619Z)

Map how the test suite runs and what committed fixture data already exists. Read `run_all_tests.py`; survey the `tests/` directory; find committed data fixtures (Glob `tests/**/*.csv`, `tests/**/*.json`, `backend/eval/**/*.csv`, `backend/eval/**/*.json`, and any `'fixtures'/'data'/'golden'` directories); read `tests/test_eval_regression.py`, `tests/test_gt_loader.py`, `tests/test_fusion_golden_telemetry.py` and any `conftest.py`. ANSWER: (1) How are tests discovered & run (pytest config, markers, what `run_all_tests.py` does)? (2) What committed fixture files already exist, their format and rough size? (3) How is the detector / WSL / video mocked in current tests? (4) Is there an existing 'load committed artifact -> run transform -> assert on output' pattern to follow? (5) Where (which directory) and how would NEW auto-discovered per-video regression fixtures + a parametrized test best slot in, matching existing conventions? Concrete paths + sizes.

2. assistant (2026-06-16T02:24:55.412Z)

I'll map out the test suite structure and fixture data. Let me start by reading the key files in parallel.

3. user (2026-06-16T02:24:55.906Z)

```
1      #!/usr/bin/env python
2      """Run the FULL test suite for the badminton-highlight-indexer.
3
4      A single, discoverable entry point so you (or a reviewer / CI) can verify the
5      whole suite before opening a PR:
6
7          python run_all_tests.py          # full suite, quiet
8          python run_all_tests.py -v      # verbose (passthrough to pytest)
9          python run_all_tests.py -k config # filter by keyword (passthrough)
10         python run_all_tests.py --cov=backend --cov-report=term-missing # coverage
11
12     Anything after the script name is passed straight through to pytest.
13
14     Notes
15     -----
16     - The suite is fully mocked (no GPU, no network, no WSL) and runs offline.
17     - The observability **reporting** tests skip cleanly if the optional `obsreport`
18       package isn't installed (they use `pytest.importorskip`). To exercise them too:
19         pip install -e ../sports-obsreport --no-build-isolation
20     - Exit code is pytest's own, so this is CI / pre-PR friendly (0 = all green).
21     """
22     from __future__ import annotations
23
24     import importlib.util
25     import subprocess
```

```

26     import sys
27
28
29     def _installed(module: str) -> bool:
30         try:
31             return importlib.util.find_spec(module) is not None
32         except (ImportError, ValueError):
33             return False
34
35
36     def main() -> int:
37         args = list(sys.argv[1:])
38         # Default to quiet unless the caller chose a verbosity.
39         if not any(a in ("-q", "-v", "-vv", "-vvv", "--quiet", "--verbose") for a in args):
40             args = ["-q", *args]
41         cmd = [sys.executable, "-m", "pytest", *args]
42
43         obs = "available" if _installed("obsreport") else "NOT installed ? reporting tests
will SKIP"
44         bar = "=" * 72
45         print(bar)
46         print(" Full test suite: " + " ".join(cmd))
47         print(f" obsreport (optional reporting dep): {obs}")
48         print(bar, flush=True)
49
50         rc = subprocess.run(cmd).returncode
51
52         print(bar)
53         print(" RESULT: " + ("PASS - all tests green" if rc == 0 else f"FAIL - pytest exit
{rc}"))
54         print(bar)
55         return rc
56
57
58     if __name__ == "__main__":
59         raise SystemExit(main())
60

```

4. user (2026-06-16T02:24:56.519Z)

```

tests\__init__.py
tests\test_eval_batch.py
tests\test_eval_training.py
tests\test_annotation_providers.py
tests\test_calibration.py
tests\test_calibrate_wasb.py
tests\test_export_wasb_segments.py
tests\test_gemini_refine.py
tests\test_calibrate_local.py
tests\test_distill_local.py
tests\test_automated_provider.py
tests\test_eval_harness.py
tests\test_hit_detector.py
tests\test_hashing.py
tests\test_rally_slicer.py
tests\test_reporting_spec_faq.py
tests\test_eval_partial_labels.py
tests\test_detector_keying.py
tests\test_eval_metrics.py
tests\test_eval_annotations.py
tests\test_gt_loader.py
tests\test_import_smoke.py
tests\test_reporting_harvest.py
tests\test_run_signals.py
tests\test_eval_classifier.py

```

tests\test_eval_regression.py
tests\test_geometry.py
tests\test_path_safety.py
tests\test_provider_gemini.py
tests\test_providers.py
tests\test_rally_seq_proto.py
tests\test_reliability_hardening.py
tests\test_reporting.py
tests\test_sport_profiles.py
tests\test_video.py
tests\test_upload_download.py
tests\test_watermark.py
tests\test_cleanup_caches.py
tests\test_experiment.py
tests\test_yolo_hybrid.py
tests\test_fusion_features.py
tests\test_rally_gate.py
tests\test_trajectory_hybrid_equivalence.py
tests\test_logging_setup.py
tests\test_rally_summary.py
tests\test_audio_features.py
tests\test_eval_stats.py
tests\test_person_detector.py
tests\test_score_calibration.py
tests\test_segmentation_metrics.py
tests\test_fusion_compare.py
tests\test_execution_policy.py
tests\test_experiment_honest.py
tests\test_job_self_scheduling.py
tests\test_run_telemetry.py
tests\test_variant_tuning.py
tests\test_fusion_golden_telemetry.py
tests\test_per_user_gate.py
tests\test_per_user_eval.py
tests\test_per_user_regression.py
tests\test_ablation.py
tests\test_database.py
tests\test_personalization_serving.py
tests\test_fusion_hybrid.py
tests\test_served_gate_a_regression.py
tests\test_promotion.py
tests\test_serve_contrast.py
tests\test_windowing.py
tests\test_api_endpoints.py
tests\test_async_jobs.py
tests\test_base.py
tests\test_job_worker_loop.py
tests\test_reingest_safety.py
tests\test_splits.py
tests\test_training_gen0.py
tests\test_validators_base.py
tests\test_wasb_runner.py
tests\test_wsl_tilde_quoting.py
tests\test_compiler.py
tests\test_gemini.py
tests\test_config_wasb.py
tests\test_tracknet_runner.py
tests\test_wasb_cache.py
tests\test_config.py
tests\test_generate_highlights.py
tests\test_tracknet_hybrid.py

5. user (2026-06-16T02:24:56.954Z)

No files found

6. user (2026-06-16T02:24:57.036Z)

No files found

7. assistant (2026-06-16T02:24:59.714Z)

No CSV/JSON found directly under tests/. Let me broaden the search for fixtures and read the key test files.

8. user (2026-06-16T02:25:00.300Z)

No files found

9. user (2026-06-16T02:25:00.861Z)

No files found

10. user (2026-06-16T02:25:01.232Z)

No files found

11. user (2026-06-16T02:25:01.748Z)

No files found

12. user (2026-06-16T02:25:01.750Z)

```
pytest.ini
.claude\worktrees\bhi-execpolicy\pytest.ini
.claude\worktrees\agent-a771ff1d85e18f2ed\pytest.ini
.claude\worktrees\local-noai\pytest.ini
```

13. user (2026-06-16T02:25:02.200Z)

```
pyproject.toml
.claude\worktrees\agent-a771ff1d85e18f2ed\pyproject.toml
.claude\worktrees\local-noai\pyproject.toml
```

14. user (2026-06-16T02:25:02.650Z)

No files found

15. user (2026-06-16T02:25:02.781Z)

No files found

16. assistant (2026-06-16T02:25:06.741Z)

There's no conftest.py and no CSV/JSON fixtures in the standard locations. Let me read the config files and the three named test files, plus search more broadly for fixture data.

17. user (2026-06-16T02:25:07.964Z)

```
1      [pytest]
2      # When `pytest` is invoked without an explicit path, only collect from tests/.
3      # scratch/ holds debugging scripts that hit real APIs (Gemini upload, etc.)
4      # and should never run as part of the regular test suite.
5      testpaths = tests
6
```

18. user (2026-06-16T02:25:08.405Z)

```
1      # PEP 621 packaging for the badminton/sports highlight indexer (DEFERRED #7).
```

```

2      #
3      # Build backend: hatchling. `backend/` is an intentional NAMESPACE package
4      # (no `backend/__init__.py`; mypy.ini sets explicit_package_bases for the same
5      # reason), so the wheel target packages are listed EXPLICITLY below ? hatchling's
6      # auto-detection would otherwise miss the un-__init__'d `backend` root.
7      #
8      # torch/torchvision are deliberately NOT listed in [project.dependencies]:
9      # their CPU/CUDA wheels need an out-of-band index (--index-url / --extra-index-url),
10     # which PEP 621 cannot express. Install them separately ? see [project.optional-
dependencies]
11     # `gpu` (documentation only) and the README. CI installs CPU torch on its own line.
12     [build-system]
13     requires = ["hatchling"]
14     build-backend = "hatchling.build"
15
16     [project]
17     name = "badminton-highlight-indexer"
18     version = "0.1.0"
19     description = "AI-assisted rally segmentation and highlight indexing for racket sports."
20     readme = "README.md"
21     requires-python = ">=3.10"
22     license = { text = "MIT" }
23     authors = [{ name = "Avi Dullu", email = "avi.dullu@gmail.com" }]
24     keywords = ["badminton", "sports", "video", "highlights", "rally", "segmentation"]
25
26     # Runtime deps mirror requirements.txt, EXCEPT torch/torchvision (see header).
27     dependencies = [
28         "fastapi>=0.111.0",
29         "uvicorn>=0.30.1",
30         "opencv-python>=4.9.0.80",
31         "numpy>=1.26.4",
32         "pydantic>=2.7.1",
33         "jinja2>=3.1.4",
34         "python-dotenv>=1.0.0",
35         "google-genai>=2.4.0",
36         "supervision>=0.21.0,<0.30.0",
37     ]
38
39     [project.optional-dependencies]
40     # Test + lint/type tooling ? matches the CI lanes (.github/workflows/ci.yml).
41     dev = [
42         "pytest",
43         "pytest-cov",
44         "httpx",          # fastapi TestClient transport
45         "ruff",
46         "mypy",
47     ]
48
49     # Per-video observability reports (SOFT dependency: the pipeline degrades to a
50     # no-op if absent). Pinned to its git tag per BACKLOG. The private repo is
51     # fetched over SSH; if you lack access, simply omit this group ? reporting
52     # tests skip via pytest.importorskip. For local dev against a sibling checkout:
53     # pip install -e ../sports-obsreport --no-build-isolation
54     reporting = [
55         "obsreport @ git+ssh://git@github.com/avidullu/sports-obsreport.git@v0.1.3",
56     ]
57
58     # `gpu` is DOCUMENTATION ONLY ? it is intentionally empty. torch/torchvision
59     # CUDA wheels require `--index-url`/`--extra-index-url`, which PEP 621 cannot
60     # express, so they are NOT declarable here. Install them out-of-band instead:
61     # # CUDA 12.4 box:
62     # pip install torch>=2.12.0 torchvision>=0.27.0 --index-url
https://download.pytorch.org/whl/cu124
63     # # CPU-only box (also what CI uses):
64     # pip install torch torchvision --index-url https://download.pytorch.org/whl/cpu

```

```

65     gpu = []
66
67     [project.scripts]
68     # `indexer --server`, `indexer --help`, etc. -> backend/main.py::main()
69     indexer = "backend.main:main"
70
71     [tool.hatch.metadata]
72     # The `reporting` extra pins obsreport to a git tag (a PEP 508 *direct
73     # reference*); hatchling requires opting in to allow those in metadata.
74     allow-direct-references = true
75
76     [tool.hatch.build.targets.wheel]
77     # `backend` is a namespace package (no __init__.py); list both import roots
78     # explicitly so an editable/wheel install exposes `import backend` and
79     # `import training` regardless of cwd.
80     packages = ["backend", "training"]
81

```

19. user (2026-06-16T02:25:09.101Z)

```

1     """Tests for the regression runner (GS-2): baseline store, regress(), the
2     provider-driven end-to-end path, and the CLI exit-code contract."""
3     import pytest
4
5     from backend.infrastructure.database import Database
6     from backend.eval import regression
7     import run_regression
8
9
10    @pytest.fixture
11    def db(tmp_path):
12        d = Database(str(tmp_path / "t.db"))
13        d.add_video("vid1", "/x/m.mp4", "processed", [])
14        # Two confirmed segments matching the GT below ? perfect baseline.
15        d.add_play_segment("vid1", 0.0, 10.0)
16        d.add_play_segment("vid1", 20.0, 30.0)
17        return d
18
19
20    GT = [(0.0, 10.0), (20.0, 30.0)]
21
22
23    # ----- baseline store -----
24
25    def test_save_and_load_baseline_roundtrip(tmp_path):
26        p = str(tmp_path / "b.json")
27        regression.save_baseline("vid1", "final", 1.0, 1.0, 1.0, path=p)
28        b = regression.load_baselines(p)
29        assert b["vid1::final"]["precision"] == 1.0
30
31
32    def test_load_baselines_absent_is_empty(tmp_path):
33        assert regression.load_baselines(str(tmp_path / "nope.json")) == {}
34
35
36    # ----- regress() -----
37
38    def test_no_baseline_is_not_a_regression(db, tmp_path):
39        r = regression.regress(db, "vid1", GT, baseline_path=str(tmp_path / "b.json"))
40        assert r.regressed is False
41        assert r.precision == 1.0 and r.recall == 1.0
42        assert any("no baseline" in why for why in r.reasons)
43
44
45    def test_matching_baseline_passes(db, tmp_path):

```

```

46     p = str(tmp_path / "b.json")
47     regression.save_baseline("vid1", "final", 1.0, 1.0, 1.0, path=p)
48     r = regression.regress(db, "vid1", GT, baseline_path=p)
49     assert r.regressed is False
50
51
52 def test_recall_drop_beyond_tolerance_regresses(tmp_path):
53     # DB has only ONE of the two GT rallies ? recall 0.5 vs baseline 1.0.
54     d = Database(str(tmp_path / "t.db"))
55     d.add_video("vid1", "/x/m.mp4", "processed", [])
56     d.add_play_segment("vid1", 0.0, 10.0)
57     p = str(tmp_path / "b.json")
58     regression.save_baseline("vid1", "final", 1.0, 1.0, 1.0, path=p)
59
60     r = regression.regress(d, "vid1", GT, tolerance=0.05, baseline_path=p)
61     assert r.regressed is True
62     assert r.recall == pytest.approx(0.5)
63     assert any("recall" in why for why in r.reasons)
64
65
66 def test_within_tolerance_does_not_regress(tmp_path):
67     d = Database(str(tmp_path / "t.db"))
68     d.add_video("vid1", "/x/m.mp4", "processed", [])
69     d.add_play_segment("vid1", 0.0, 10.0)
70     d.add_play_segment("vid1", 20.0, 30.0)
71     p = str(tmp_path / "b.json")
72     # Baseline only 0.02 above current perfect score ? within default 0.05 tolerance.
73     regression.save_baseline("vid1", "final", 1.0, 1.0, 1.0, path=p)
74     r = regression.regress(d, "vid1", GT, tolerance=0.05, baseline_path=p)
75     assert r.regressed is False
76
77
78 # ----- provider-driven end-to-end (uses the GS-1 FileProvider) -----
79
80 def test_regress_with_provider_file_tier(db, tmp_path):
81     csv = tmp_path / "gt.csv"
82     csv.write_text("start,end\n0,10\n20,30\n")
83     r = regression.regress_with_provider(
84         db, video_id="vid1", video_ref=str(csv), tier="file",
85         baseline_path=str(tmp_path / "b.json"),
86     )
87     assert r.precision == 1.0 and r.recall == 1.0
88
89
90 # ----- CLI exit-code contract (CI gating) -----
91
92 def test_cli_update_baseline_then_clean_run(db, tmp_path, capsys, monkeypatch):
93     # Point the runner at our temp DB via --db; CSV ground truth via 'file' tier.
94     csv = tmp_path / "gt.csv"
95     csv.write_text("start,end\n0,10\n20,30\n")
96     base = str(tmp_path / "b.json")
97     db_path = db.db_path if hasattr(db, "db_path") else str(tmp_path / "t.db")
98
99     # First, snapshot the baseline ? exit 0.
100    rc = run_regression.main(["--video-id", "vid1", "--tier", "file",
101                             "--annotations", str(csv), "--db", db_path,
102                             "--baseline", base, "--update-baseline"])
103    assert rc == 0
104    assert "REGRESSED" not in capsys.readouterr().out
105
106
107 def test_cli_regression_exits_nonzero(tmp_path, capsys):
108     # DB missing one rally vs a baseline of perfect ? regression ? exit 1.
109     d = Database(str(tmp_path / "t.db"))
110     d.add_video("vid1", "/x/m.mp4", "processed", [])

```

```

111     d.add_play_segment("vid1", 0.0, 10.0)
112     csv = tmp_path / "gt.csv"
113     csv.write_text("start,end\n0,10\n20,30\n")
114     base = str(tmp_path / "b.json")
115     regression.save_baseline("vid1", "final", 1.0, 1.0, 1.0, path=base)
116
117     rc = run_regression.main(["--video-id", "vid1", "--tier", "file",
118                             "--annotations", str(csv), "--db", str(tmp_path / "t.db"),
119                             "--baseline", base])
120     assert rc == 1
121     assert "REGRESSED" in capsys.readouterr().out
122
123
124     def test_cli_missing_baseline_exits_3(db, tmp_path):
125         # No baseline + no --update/--allow ? the gate cannot verify ? exit 3 (not a silent
126         pass).
127         csv = tmp_path / "gt.csv"
128         csv.write_text("start,end\n0,10\n20,30\n")
129         db_path = str(tmp_path / "t.db")
130         rc = run_regression.main(["--video-id", "vid1", "--tier", "file",
131                                 "--annotations", str(csv), "--db", db_path,
132                                 "--baseline", str(tmp_path / "absent.json")])
133         assert rc == 3
134
135     def test_cli_missing_baseline_allowed_exits_0(db, tmp_path):
136         csv = tmp_path / "gt.csv"
137         csv.write_text("start,end\n0,10\n20,30\n")
138         rc = run_regression.main(["--video-id", "vid1", "--tier", "file",
139                                 "--annotations", str(csv), "--db", str(tmp_path / "t.db"),
140                                 "--baseline", str(tmp_path / "absent.json"),
141                                 "--allow-missing-baseline"])
142         assert rc == 0
143
144
145     def test_iou_mismatch_is_incommensurable(db, tmp_path):
146         p = str(tmp_path / "b.json")
147         regression.save_baseline("vid1", "final", 1.0, 1.0, 1.0, path=p, iou_threshold=0.5,
148                                gt_fingerprint=regression.gt_hash(GT))
149         # Same labels but scored at a different IoU ? not a valid comparison.
150         r = regression.regress(db, "vid1", GT, iou_threshold=0.3, baseline_path=p)
151         assert r.regressed is True
152         assert any("incommensurable" in why for why in r.reasons)
153
154
155     def test_changed_labels_detected_via_gt_hash(db, tmp_path):
156         p = str(tmp_path / "b.json")
157         regression.save_baseline("vid1", "final", 1.0, 1.0, 1.0, path=p, iou_threshold=0.5,
158                                gt_fingerprint=regression.gt_hash(GT))
159         r = regression.regress(db, "vid1", [(0.0, 10.0), (21.0, 31.0)], iou_threshold=0.5,
160                                baseline_path=p)
161         assert r.regressed is True
162         assert any("labels changed" in why for why in r.reasons)

```

20. user (2026-06-16T02:25:09.596Z)

```

1     """Canonical GT loader: one reader for BOTH golden-CSV conventions (ADR-008 prereq).
2
3     Pins the frozen golden-label contract: the collector's `curate export` format
4     (`start,end`) and the rally-annotator golden format
5     (`rally_number,start_time,end_time,...`)
6     must load IDENTICALLY through every GT entry point ? the historical two-loader fork
7     (zero/garbage intervals when formats crossed) must never reopen.
8     """

```



```

8     import csv
9
10    import pytest
11
12    from backend.eval.gt_loader import load_gt_intervals
13    from backend.eval.annotations import load_annotations
14    from backend.eval.calibrate_wasb import load_golden_gts
15
16    EXPECTED = [(5.654, 8.425), (12.928, 18.673), (31.171, 39.173)]
17
18
19    def _collector_export(path):
20        """Replicates EXACTLY what the collector's export_eval_set writes (curate.py)."""
21        with open(path, "w", newline="", encoding="utf-8") as f:
22            w = csv.writer(f)
23            w.writerow(["start", "end"])
24            for s, e in EXPECTED:
25                w.writerow([s, e])
26        return str(path)
27
28
29    def _annotator_golden(path):
30        """Replicates the rally-annotator .rallies.csv schema."""
31        with open(path, "w", newline="", encoding="utf-8") as f:
32            w = csv.writer(f)
33            w.writerow(["rally_number", "start_time", "end_time", "ending_reason", "sport"])
34            for i, (s, e) in enumerate(EXPECTED, 1):
35                w.writerow([i, s, e, "winner", "badminton"])
36        return str(path)
37
38
39    @pytest.mark.parametrize("loader", [load_gt_intervals, load_annotations,
load_golden_gts])
40    def test_collector_export_format_loads_everywhere(tmp_path, loader):
41        p = _collector_export(tmp_path / "export.csv")
42        assert loader(p) == EXPECTED
43
44
45    @pytest.mark.parametrize("loader", [load_gt_intervals, load_annotations,
load_golden_gts])
46    def test_annotator_golden_format_loads_everywhere(tmp_path, loader):
47        p = _annotator_golden(tmp_path / "golden.csv")
48        assert loader(p) == EXPECTED
49
50
51    def test_round_trip_export_equals_golden(tmp_path):
52        """The fork regression: both conventions of the SAME rallies -> identical
intervals."""
53        a = load_gt_intervals(_collector_export(tmp_path / "a.csv"))
54        b = load_gt_intervals(_annotator_golden(tmp_path / "b.csv"))
55        assert a == b == EXPECTED
56
57
58    def test_headerless_positional_and_mmss(tmp_path):
59        p = tmp_path / "h.csv"
60        p.write_text("# comment\n00:01:12,00:01:39\n102.5,131.0\n", encoding="utf-8")
61        assert load_gt_intervals(str(p)) == [(72.0, 99.0), (102.5, 131.0)]
62
63
64    def test_strict_raises_with_location(tmp_path):
65        p = tmp_path / "bad.csv"
66        p.write_text("start,end\n10,5\n", encoding="utf-8") # reversed
67        with pytest.raises(ValueError, match="bad.csv:2"):
68            load_gt_intervals(str(p), strict=True)
69        p2 = tmp_path / "neg.csv"

```

```

70         p2.write_text("start,end\n-1,5\n", encoding="utf-8")
71         with pytest.raises(ValueError, match="negative"):
72             load_gt_intervals(str(p2), strict=True)
73
74
75     def test_lenient_skips_and_logs(tmp_path, caplog):
76         p = tmp_path / "mixed.csv"
77
78         p.write_text("rally_number,start_time,end_time\n1,1.0,2.0\n2,,\n3,9.0,4.0\n4,5.0,6.0\n",
79                     encoding="utf-8")
80         with caplog.at_level("WARNING"):
81             out = load_gt_intervals(str(p), strict=False)
82             assert out == [(1.0, 2.0), (5.0, 6.0)]
83             assert "skipped 2" in caplog.text
84
85     def test_overlap_warning(tmp_path, caplog):
86         p = tmp_path / "ov.csv"
87         p.write_text("start,end\n1,10\n5,12\n", encoding="utf-8")
88         with caplog.at_level("WARNING"):
89             out = load_gt_intervals(str(p))
90             assert len(out) == 2
91             assert "overlapping" in caplog.text.lower()
92

```

21. user (2026-06-16T02:25:10.342Z)

```

1     """GPU-free tests for the run-telemetry wire-in to the fusion_golden extractor.
2
3     `run()` builds a `RunTelemetry` black box in `out_dir` and emits video_start /
4     video_done events
5     around each extract_video; a hard kill during a long GPU run thus leaves a forensic
6     runlog. These
7     tests stub extract_video (no GPU) and inject deterministic collectors + clock via a
8     patched
9     RunTelemetry factory, so the JSONL content is asserted with no real nvidia-smi / psutil
10    / wall-clock.
11    A telemetry-collector failure must NOT break extraction (graceful degradation).
12    """
13    import functools
14    import json
15    import os
16
17    from backend.eval import fusion_golden
18    from backend.utils import run_telemetry as rt
19
20    class _Clock:
21        def __init__(self, t=1000.0):
22            self.t = t
23
24        def __call__(self):
25            return self.t
26
27        def tick(self, dt):
28            self.t += dt
29
30    def _fake_gpu():
31        return {"temp_c": 70.0, "util_pct": 80.0, "throttled": False}
32
33    def _fake_sys():
34        return {"cpu_pct": 50.0, "ram_used_gb": 21.0}
35

```

```

35
36     def _patch_telemetry(monkeypatch, gpu_fn=_fake_gpu, sys_fn=_fake_sys):
37         """Make run()'s `RunTelemetry(...)` hermetic: inject deterministic collectors +
clock and a
38         fixed GPU-name header (no real nvidia-smi). Returns nothing; run() builds the
instance itself."""
39         monkeypatch.setattr(rt, "gpu_name", lambda: "FakeGPU")
40         factory = functools.partial(rt.RunTelemetry, gpu_fn=gpu_fn, sys_fn=sys_fn,
clock=_Clock())
41         monkeypatch.setattr(fusion_golden, "RunTelemetry", factory)
42
43
44     def _write_manifest(tmp_path, names):
45         manifest = [
46             {"name": n, "trajectory": f"{n}.csv", "fps": 30.0, "frame_width": 1920.0,
47              "labels": f"{n}.rallies.csv"}
48             for n in names
49         ]
50         mp = str(tmp_path / "m.json")
51         with open(mp, "w", encoding="utf-8") as f:
52             json.dump(manifest, f)
53         return mp
54
55
56     def _read_lines(path):
57         with open(path, encoding="utf-8") as f:
58             return [ln for ln in f.read().splitlines() if ln]
59
60
61     def test_run_writes_telemetry_with_start_and_done_events(tmp_path, monkeypatch):
62         """run() drops run_telemetry.jsonl in out_dir: a meta header +
video_start/video_done lines."""
63         _patch_telemetry(monkeypatch)
64         out_dir = str(tmp_path / "out")
65         os.makedirs(out_dir)
66
67         def _stub(spec, **kw):
68             # A real telemetry instance is threaded through; drive one snapshot from inside
extraction.
69             telem = kw.get("telemetry")
70             if telem is not None:
71                 telem.snapshot(5, 10, phase=spec["name"])
72             return {"name": spec["name"], "fps": 30.0, "frame_width": 1920.0,
73                    "candidates": [{"label": 1}, {"label": 0}], "gts": []}
74
75         monkeypatch.setattr(fusion_golden, "extract_video", _stub)
76
77         fusion_golden.run(_write_manifest(tmp_path, ["alpha", "beta"]), out_dir)
78
79         rt_path = os.path.join(out_dir, "run_telemetry.jsonl")
80         assert os.path.isfile(rt_path)
81         lines = _read_lines(rt_path)
82         recs = [json.loads(ln) for ln in lines]
83         # meta header is always the first line.
84         assert recs[0]["meta"]["label"] == "fusion_golden"
85         events = [r for r in recs if "event" in r]
86         starts = [r for r in events if r["event"] == "video_start"]
87         dones = [r for r in events if r["event"] == "video_done"]
88         assert {r["msg"] for r in starts} == {"alpha", "beta"}
89         # video_done carries the per-video window/positive counts for the post-mortem.
90         done_by = {r["msg"]: r for r in dones}
91         assert done_by["alpha"]["windows"] == 2 and done_by["alpha"]["positives"] == 1
92         # health is attached to every event (the machine state at each milestone).
93         assert starts[0]["gpu"]["temp_c"] == 70.0 and starts[0]["sys"]["cpu_pct"] == 50.0
94         # every body line is valid JSON.

```

```

95         for ln in lines:
96             json.loads(ln)
97
98
99     def test_run_records_error_event_then_reraises(tmp_path, monkeypatch):
100         """A death inside extract_video records an `error` event (with machine state) then
re-raises."""
101         _patch_telemetry(monkeypatch)
102         out_dir = str(tmp_path / "out")
103         os.makedirs(out_dir)
104
105         def _boom(spec, **kw):
106             raise RuntimeError("GPU vanished")
107
108         monkeypatch.setattr(fusion_golden, "extract_video", _boom)
109
110         raised = False
111         try:
112             fusion_golden.run(_write_manifest(tmp_path, ["alpha"]), out_dir)
113         except RuntimeError as e:
114             raised = True
115             assert "GPU vanished" in str(e)
116         assert raised, "extract_video failure must propagate"
117
118         recs = [json.loads(ln) for ln in _read_lines(os.path.join(out_dir,
"run_telemetry.jsonl"))]
119         errs = [r for r in recs if r.get("event") == "error"]
120         assert len(errs) == 1
121         assert errs[0]["msg"] == "alpha" and "GPU vanished" in errs[0]["err"]
122         assert errs[0]["gpu"]["temp_c"] == 70.0 # machine state captured at the moment of
death
123
124
125     def test_telemetry_collector_failure_does_not_break_run(tmp_path, monkeypatch):
126         """A raising GPU collector degrades gracefully ? extraction still completes and
writes output."""
127         def _exploding_gpu():
128             raise RuntimeError("nvidia-smi exploded")
129
130         _patch_telemetry(monkeypatch, gpu_fn=_exploding_gpu, sys_fn=lambda: None)
131         out_dir = str(tmp_path / "out")
132         os.makedirs(out_dir)
133
134         def _stub(spec, **kw):
135             telem = kw.get("telemetry")
136             if telem is not None:
137                 telem.snapshot(1, 10, phase=spec["name"]) # must not raise despite the bad
collector
138                 return {"name": spec["name"], "fps": 30.0, "frame_width": 1920.0,
139                     "candidates": [], "gts": []}
140
141         monkeypatch.setattr(fusion_golden, "extract_video", _stub)
142
143         written = fusion_golden.run(_write_manifest(tmp_path, ["alpha"]), out_dir)
144
145         assert written == [os.path.join(out_dir, "alpha_golden_features.json")]
146         assert os.path.isfile(written[0])
147         recs = [json.loads(ln) for ln in _read_lines(os.path.join(out_dir,
"run_telemetry.jsonl"))]
148         # The run survived; events still recorded, just without a gpu/sys block.
149         assert any(r.get("event") == "video_done" for r in recs)
150         assert all("gpu" not in r for r in recs if "event" in r)
151
152
153     def test_run_survives_telemetry_construction_failure(tmp_path, monkeypatch):

```

```

154         """If RunTelemetry construction itself raises, run() must still extract (telemetry
is best-effort)."""
155         def _boom_ctor(*a, **k):
156             raise RuntimeError("cannot build telemetry")
157
158         monkeypatch.setattr(fusion_golden, "RunTelemetry", _boom_ctor)
159         out_dir = str(tmp_path / "out")
160         os.makedirs(out_dir)
161
162         def _stub(spec, **kw):
163             assert kw.get("telemetry") is None # graceful fallback to no telemetry
164             return {"name": spec["name"], "fps": 30.0, "frame_width": 1920.0,
165                     "candidates": [], "gts": []}
166
167         monkeypatch.setattr(fusion_golden, "extract_video", _stub)
168
169         written = fusion_golden.run(_write_manifest(tmp_path, ["alpha"]), out_dir)
170         assert written == [os.path.join(out_dir, "alpha_golden_features.json")]
171

```

22. assistant (2026-06-16T02:25:15.461Z)

Key finding: tests build all their fixtures inline (tmp_path), no committed CSV/JSON. Let me look for committed data fixtures across the whole repo, examine the `backend/eval` module structure, and find any existing per-video golden/manifest pattern. Let me also look at the fusion_golden module and gt_loader.

23. user (2026-06-16T02:25:18.283Z)

```

config.json
eval_baselines/experiment_leaderboard.json
eval_baselines/gemini_wrapper_2026-06-10.json
eval_baselines/heuristic_lovo_n6.json

```

24. user (2026-06-16T02:25:18.406Z)

```

backend\eval\__init__.py
backend\eval\classifier.py
backend\eval\training.py
backend\eval\gemini_refine.py
backend\eval\audio\__init__.py
backend\eval\audio\e1_probe.py
backend\eval\export_wasb_segments.py
backend\eval\metrics.py
backend\eval\regression.py
backend\eval\annotations.py
backend\eval\calibrate_wasb.py
backend\eval\calibration.py
backend\eval\eval_partial_labels.py
backend\eval\rally_seq_proto.py
backend\eval\calibrate_local.py
backend\eval\gt_loader.py
backend\eval\harness.py
backend\eval\eval_stats.py
backend\eval\score_calibration.py
backend\eval\segmentation_metrics.py
backend\eval\fusion_audio.py
backend\eval\fusion_compare.py
backend\eval\experiment.py
backend\eval\batch.py
backend\eval\fusion_golden.py
backend\eval\per_user_gate.py
backend\eval\per_user_eval.py
backend\eval\per_user_regression.py

```

backend\eval\ablation.py
backend\eval\served_gate_a.py
backend\eval\distill_local.py
backend\eval\promotion.py
backend\eval\serve_contrast.py
backend\eval>windowing.py
backend\eval\splits.py

25. user (2026-06-16T02:25:19.173Z)

backend\eval__init__.py
backend\eval\classifier.py
backend\eval\training.py
backend\providers__init__.py
backend\sports__init__.py
backend\sports\badminton.py
backend\sports\base.py
backend\utils__init__.py
backend\utils\geometry.py
scratch\analyze_motion.py
scratch\clean_state.py
scratch\diagnose_rally.py
scratch\gemini_video_debugger.py
scratch\list_models.py
scratch\test_thresholds.py
tests__init__.py
tests\test_eval_batch.py
tests\test_eval_training.py
backend\annotations\base.py
backend\annotations\file_provider.py
tests\test_annotation_providers.py
backend\annotations\automated_provider.py
backend\annotations__init__.py
backend\tools\rally_slicer.py
backend\tools__init__.py
tests\test_calibration.py
tests\test_calibrate_wasb.py
backend\eval\gemini_refine.py
tests\test_export_wasb_segments.py
tests\test_gemini_refine.py
tests\test_calibrate_local.py
tests\test_distill_local.py
backend\eval\audio__init__.py
backend\eval\audio\e1_probe.py
backend\eval\export_wasb_segments.py
backend\pipeline\audio__init__.py
backend\pipeline\validators__init__.py
scratch\test_ai_segmenter.py
tests\test_automated_provider.py
tests\test_eval_harness.py
tests\test_hit_detector.py
run_all_tests.py
backend\results.py
run_eval.py
tests\test_hashing.py
tests\test_rally_slicer.py
tests\test_reporting_spec_faq.py
tests\test_eval_partial_labels.py
tests\test_detector_keying.py
backend\eval\metrics.py
backend\eval\regression.py
run_regression.py
tests\test_eval_metrics.py
backend\eval\annotations.py
tests\test_eval_annotations.py

```

tests\test_gt_loader.py
backend\features\__init__.py
tests\test_import_smoke.py
training\gen0\__init__.py
training\__init__.py
backend\reporting\harvest.py
backend\reporting\run_signals.py
tests\test_reporting_harvest.py
tests\test_run_signals.py
backend\eval\calibrate_wasb.py
backend\eval\calibration.py
backend\eval\eval_partial_labels.py
backend\eval\rally_seq_proto.py
backend\features\rally_seq.py
backend\pipeline\detectors\__init__.py
backend\pipeline\segmenters\base.py
backend\pipeline\segmenters\motion.py
backend\pipeline\validators\blur.py
backend\pipeline\validators\format.py
backend\pipeline\validators\pts_continuity.py
backend\pipeline\validators\relevance.py
backend\pipeline\validators\resolution_fps.py
backend\pipeline\validators\scene_cut.py
backend\providers\base.py
backend\reporting\__init__.py
tests\test_eval_classifier.py
tests\test_eval_regression.py
tests\test_geometry.py
tests\test_path_safety.py
tests\test_provider_gemini.py
tests\test_providers.py
tests\test_rally_seq_proto.py
tests\test_reliability_hardening.py
tests\test_reporting.py
tests\test_sport_profiles.py
tests\test_video.py
backend\eval\calibrate_local.py
backend\eval\gt_loader.py
backend\eval\harness.py
backend\pipeline\audio\hit_detector.py
backend\pipeline\validators\base.py
backend\utils\paths.py
backend\utils\video.py
backend\pipeline\segmenters\tracknet_hybrid.py
backend\pipeline\segmenters\wasb_hybrid.py
(Results are truncated. Consider using a more specific path or pattern.)

```

26. assistant (2026-06-16T02:25:22.528Z)

Now I have the committed JSON fixtures. Let me examine the `eval\_baselines/` directory (committed baseline JSONs), the fusion_golden module (which uses manifest+CSV per-video), and the gt_loader. Let me also check the size of those committed JSON files and how the detector/WSL/video is mocked.

27. user (2026-06-16T02:25:24.646Z)

```
-rw-r--r-- 1 avidu 197609 2735 Jun 16 01:52 config.json
```

```
eval_baselines/:
```

```
total 28
```

```
drwxr-xr-x 1 avidu 197609    0 Jun 14 16:28 .
```

```
drwxr-xr-x 1 avidu 197609    0 Jun 16 01:52 ..
```

```
-rw-r--r-- 1 avidu 197609 4696 Jun 14 16:28 experiment_leaderboard.json
```

```
-rw-r--r-- 1 avidu 197609 1977 Jun 10 17:43 gemini_wrapper_2026-06-10.json
```

```
-rw-r--r-- 1 avidu 197609  689 Jun 10 15:57 heuristic_lovo_n6.json
```

---WC---

```
85 config.json
139 eval_baselines/experiment_leaderboard.json
24 eval_baselines/gemini_wrapper_2026-06-10.json
15 eval_baselines/heuristic_lovo_n6.json
263 total
```

---GITLS eval_baselines---

```
eval_baselines/experiment_leaderboard.json
eval_baselines/gemini_wrapper_2026-06-10.json
eval_baselines/heuristic_lovo_n6.json
```

28. user (2026-06-16T02:25:24.838Z)

```
1      """Offline golden-set person-feature extractor ? the fusion-P3 measurement's GPU step.
2
3      Mirrors `distill_local`'s CSV-driven flow but ADDS the ~5 person-cue features per
shuttle
4      candidate window, by running `PersonDetector` over the ORIGINAL video. It reuses the
5      EXISTING golden trajectory CSVs (so **no WASB re-run**), needs **no Gemini and no DB**,
and
6      writes one small JSON per video. The LOVO comparison (`fusion_compare.py`) then runs
100% on
7      CPU offline. So the GPU touches exactly one thing ? person detection ? and the lift is
8      measured by the shipped experiment harness; person features enter only as feature
columns the
9      learned scorer weights (no special-casing, default-OFF until the golden ?F1 justifies
them).
10
11     Run (on the GPU box):
12         python -m backend.eval.fusion_golden --manifest output/human_lovo_manifest.json
--out-dir output
13
14     Each output `output/<name>_golden_features.json` is the replayable feature substrate ?
re-run
15     `fusion_compare` with different variants/thresholds forever without re-detecting on the
GPU.
16     """
17     from __future__ import annotations
18
19     import argparse
20     import json
21     import os
22     import time
23     from typing import Any, Dict, List, Optional
24
25     from backend.pipeline.detectors.tracknet_runner import TrackNetRunner
26     from backend.pipeline.detectors import rally_gate
27     from backend.pipeline.detectors import fusion_features as ff
28     from backend.eval import distill_local
29     from backend.eval.calibrate_wasb import load_golden_gts
30     from backend.eval.training import label_candidates
31     from backend.utils.run_telemetry import RunTelemetry
32
33     # The 5 fps/resolution-invariant shuttle features distill_local already computes.
34     SHUTTLE_FEATURE_NAMES = list(distill_local.FEATURE_NAMES)
35     # The 5 person-cue features (fusion_features.PERSON_FEATURE_NAMES).
36     PERSON_FEATURE_NAMES = list(ff.PERSON_FEATURE_NAMES)
37
38
39     def _derive_video_path(spec: Dict[str, Any]) -> str:
40         """Original MP4 for a manifest entry ? explicit `video_path`, else derived from the
41         golden labels path (`X.rallies.csv` -> `X.MP4`, the golden-set naming
convention)."""
42         if spec.get("video_path"):
43             return str(spec["video_path"])
```


[illegible]

```

100
101 candidates: List[Dict[str, Any]] = []
102 for i, ((s, e), xs) in enumerate(zip(intervals, x_shuttle)):
103     det = per_window[i]
104     det_fw = det.get("frame_width") or fw
105     det_fh = det.get("frame_height") or 0.0
106     # Normalise the (pixel) net coord to 0-1 on the play axis, matching the foot-
point
107     # normalisation fusion_features uses (x/width, y/height).
108     if axis == "x":
109         net_norm = net_px / det_fw if det_fw > 0 else 0.5
110     else:
111         net_norm = net_px / det_fh if det_fh > 0 else 0.5
112     sf, ef = win_frames[i]
113     shuttle_in = [p for p in points if sf <= p.frame <= ef]
114     person = ff.compute_person_features(det["frames"], shuttle_in, det_fw, det_fh,
115                                         court_polygon=None, net=(axis, net_norm))
116     candidates.append({
117         "start": float(s), "end": float(e),
118         "shuttle": {k: float(v) for k, v in zip(SHUTTLE_FEATURE_NAMES, xs)},
119         "person": {k: float(v) for k, v in person.items()},
120         "label": int(labels[i]),
121     })
122     return {
123         "name": spec["name"], "fps": fps, "frame_width": fw,
124         "candidates": candidates,
125         "gts": [[float(a), float(b)] for a, b in gts],
126     }
127
128
129 def run(manifest_path: str, out_dir: str, frame_skip: int = 3, iou: float = 0.5,
130        fresh: bool = False, smallest_first: bool = False) -> List[str]:
131     with open(manifest_path, encoding="utf-8") as f:
132         specs = json.load(f)
133     if smallest_first:
134         # Process small videos first for quick feedback (the slow full match lands
last);
135         # the LOVO result is order-independent. Proxy size by the trajectory CSV (one
row/frame).
136         specs = sorted(specs, key=lambda s: (os.path.getsize(s["trajectory"])
137                                             if os.path.isfile(s.get("trajectory", ""))
else 0))
138     os.makedirs(out_dir, exist_ok=True)
139     # Run-telemetry black box: a long GPU run can be OS-killed with no traceback, so
keep a tiny
140     # rotating runlog of velocity + machine health for the post-mortem. Constructing it
must never
141     # break extraction, so guard the build and fall back to no telemetry on any error.
142     telem: Optional[RunTelemetry]
143     try:
144         telem = RunTelemetry(os.path.join(out_dir, "run-telemetry.jsonl"),
label="fusion_golden")
145     except Exception: # noqa: BLE001 - telemetry wiring must never break the run it
observes
146         telem = None
147     written: List[str] = []
148     for vi, spec in enumerate(specs, 1):
149         out = os.path.join(out_dir, f"{spec['name']}_golden_features.json")
150         # Resumable: skip a video whose JSON already exists (a re-run picks up where a
prior
151         # run left off instead of re-detecting). --fresh forces a redo.
152         if not fresh and os.path.isfile(out):
153             print(f"[fusion_golden] ({vi}/{len(specs)}) {spec['name']}: already
extracted "
154                 f"-> {out} (use --fresh to redo)", flush=True)

```

```

155         written.append(out)
156         continue
157     print(f"[fusion_golden] ({vi}/{len(specs)}) {spec['name']}: extracting...",
flush=True)
158     t0 = time.monotonic()
159     if telem is not None:
160         telem.event("video_start", spec["name"])
161     try:
162         data = extract_video(spec, frame_skip=frame_skip, iou=iou, telemetry=telem)
163     except Exception as e: # noqa: BLE001 - record machine state at death, then re-
raise
164         if telem is not None:
165             telem.event("error", spec["name"], err=str(e))
166         raise
167     n_pos = sum(c["label"] for c in data["candidates"])
168     with open(out, "w", encoding="utf-8") as f:
169         json.dump(data, f, indent=2)
170     if telem is not None:
171         telem.event("video_done", spec["name"],
172                     windows=len(data["candidates"]), positives=n_pos)
173     print(f"[fusion_golden] ({vi}/{len(specs)}) {spec['name']}: DONE in
{time.monotonic() - t0:.0f}s "
174         f"| {len(data['candidates'])} windows ({n_pos} positive) |
{len(data['gts'])} golden rallies "
175         f"-> {out}", flush=True)
176     written.append(out)
177     return written
178
179
180 def main() -> None:
181     ap = argparse.ArgumentParser(description="Extract golden-set shuttle+person features
(fusion P3 GPU step).")
182     ap.add_argument("--manifest", default="output/human_lovo_manifest.json")
183     ap.add_argument("--out-dir", default="output")
184     ap.add_argument("--frame-skip", type=int, default=3)
185     ap.add_argument("--iou", type=float, default=0.5)
186     ap.add_argument("--fresh", action="store_true", help="re-extract even if a JSON
already exists")
187     ap.add_argument("--smallest-first", action="store_true",
188                     help="process smaller videos first for quick feedback (LOVO is
order-independent)")
189     args = ap.parse_args()
190     run(args.manifest, args.out_dir, args.frame_skip, args.iou, fresh=args.fresh,
191         smallest_first=args.smallest_first)
192
193
194 if __name__ == "__main__":
195     main()
196

```

29. assistant (2026-06-16T02:25:39.192Z)

This gives me the per-video golden-feature JSON format. Now let me look at how the detector/WSL/video is mocked. Let me check the WASB runner test, TrackNet runner test, video test, and the fusion_compare module (which consumes the golden JSON). Let me also read the committed JSON fixtures.

30. user (2026-06-16T02:25:40.253Z)

```

1      {
2      "name": "heuristic_velocity_windowing_LOVO",
3      "description": "The honest cross-video floor the Gen-0 learned model must beat:
calibrated velocity-windowing heuristic, leave-one-video-out over the n=6 owned golden corpus.",
4      "computed": "2026-06-10, reproduced via: python -m backend.eval.calibrate_local
--manifest output/human_lovo_manifest.json --metric f1 (matches docs/archives/quality-

```

```

iterations/2026-06-multivideo-lovo)",
5      "mean_f1": 0.480,
6      "std": 0.242,
7      "per_video": {
8          "adarsh_avi_singles": 0.751,
9          "kushagra_singles": 0.157,
10         "largetest_doubles": 0.737,
11         "gbaaddy": 0.309,
12         "testlarge_short": 0.267,
13         "mahadevpura_singles": 0.657
14     }
15 }
16

```

31. user (2026-06-16T02:25:40.976Z)

```

1      [
2      {
3          "timestamp": "2026-06-14T06:23:51.507393+00:00",
4          "iou": 0.5,
5          "label_set_hash": "50d98ffbc370",
6          "num_videos": 6,
7          "variant_a": {
8              "name": "shuttle_only",
9              "spec_hash": "47c867c36f2f",
10             "aggregate": {
11                 "precision": 0.31807603780927995,
12                 "recall": 0.3101947096042718,
13                 "f1": 0.3068611057861412,
14                 "mean_iou": 0.5791831889148886
15             }
16         },
17         "variant_b": {
18             "name": "shuttle_person_fusion",
19             "spec_hash": "253f4709b4c4",
20             "aggregate": {
21                 "precision": 0.3617228001510626,
22                 "recall": 0.2708939086387243,
23                 "f1": 0.2863398846047612,
24                 "mean_iou": 0.5690111885714347
25             }
26         },
27         "delta": {
28             "precision": 0.04364676234178266,
29             "recall": -0.0393008009655475,
30             "f1": -0.02052122118138,
31             "mean_iou": -0.01017200034345389
32         },
33         "honest_delta_f1": {
34             "mean_delta": -0.02052122118137999,
35             "ci_low": -0.16462806145542527,
36             "ci_high": 0.16224446866060574,
37             "n": 6.0,
38             "ci_excludes_zero": false,
39             "sign_test": {
40                 "wins": 2.0,
41                 "losses": 4.0,
42                 "ties": 0.0,
43                 "n_effective": 6.0,
44                 "p_value": 0.6875
45             }
46         }
47     },
48     {
49         "timestamp": "2026-06-14T06:24:56.487711+00:00",

```

```

50     "iou": 0.5,
51     "label_set_hash": "50d98ffbc370",
52     "num_videos": 6,
53     "variant_a": {
54         "name": "shuttle_person",
55         "spec_hash": "ee89f1e41604",
56         "aggregate": {
57             "precision": 0.3617228001510626,
58             "recall": 0.2708939086387243,
59             "f1": 0.2863398846047612,
60             "mean_iou": 0.5690111885714347
61         }
62     },
63     "variant_b": {
64         "name": "shuttle_person_audio",
65         "spec_hash": "d68c39eba87c",
66         "aggregate": {
67             "precision": 0.4530798177857001,
68             "recall": 0.3408138120839734,
69             "f1": 0.3655826552797436,
70             "mean_iou": 0.7074802852140344
71         }
72     },
73     "delta": {
74         "precision": 0.0913570176346375,
75         "recall": 0.06991990344524907,
76         "f1": 0.07924277067498242,
77         "mean_iou": 0.13846909664259965
78     },
79     "honest_delta_f1": {
80         "mean_delta": 0.07924277067498244,
81         "ci_low": -0.053508315003560535,
82         "ci_high": 0.2029045239922802,
83         "n": 6.0,
84         "ci_excludes_zero": false,
85         "sign_test": {
86             "wins": 4.0,
87             "losses": 2.0,
88             "ties": 0.0,
89             "n_effective": 6.0,
90             "p_value": 0.6875
91         }
92     }
93 },
94 {
95     "timestamp": "2026-06-14T00:00:00+00:00",
96     "iou": 0.5,
97     "label_set_hash": "50d98ffbc370",
98     "num_videos": 6,
99     "promotion": {
100         "cue": "gate_a_net_crossings",
101         "promoted_to": "served_default",
102         "decision": "PROMOTED ? Gate A (net_crossings>=2) flipped to the live served
default",
103         "reverted": "reverted (eval?serve): the #146 served flip was REVERTED ? on the
served COARSE windowing Gate A is neutral/negative (mean served ?F1 -0.008, ~12 real rallies
dropped), confirmed by the #148 served litmus + an independent served check. The +0.074 below is
real only on the harness's recall-oriented PROPOSAL windowing; it does NOT transfer to the
served operating point. Restored config.json default_segmenter=gemini and
FusionConfig.min_crossings default=0. Gate A stays opt-in (indexing.fusion.min_crossings=2).",
104         "served_change": "config.json indexing.default_segmenter gemini->fusion_hybrid;
FusionConfig.min_crossings default 0->2",
105         "reversible": "indexing.fusion.min_crossings=0 (Gate-A rollback) or
indexing.default_segmenter=wasb_hybrid (full revert)",
106         "gate_b": "stays OFF (min_players=0, person cue OFF)",

```

```

107         "owner_approved": true,
108         "real_video_corroboration": "GX010125_1080p offline re-score: 45 candidates ->
CONTROL 45 rallies vs Gate-A 27 (dropped 18 low-crossing held-shuttle/feed windows, ~1.7min)",
109         "over_seg_segment_count_ratio": {"before": 1.68, "after": 1.01},
110         "source_archive": "docs/archives/quality-iterations/2026-06-first-ab-
experiment/README.md"
111     },
112     "variant_a": {
113         "name": "control_shuttle_proposals",
114         "spec_hash": "control",
115         "aggregate": {}
116     },
117     "variant_b": {
118         "name": "gate_a_min_crossings_2",
119         "spec_hash": "gateA_mc2",
120         "aggregate": {}
121     },
122     "delta": {
123         "f1": 0.074
124     },
125     "honest_delta_f1": {
126         "mean_delta": 0.074,
127         "ci_low": 0.042,
128         "ci_high": 0.105,
129         "n": 6.0,
130         "ci_excludes_zero": true,
131         "sign_test": {
132             "wins": 6.0,
133             "losses": 0.0,
134             "ties": 0.0,
135             "n_effective": 6.0,
136             "p_value": 0.031
137         }
138     }
139 }
140 ]

```

32. user (2026-06-16T02:25:41.264Z)

```

1     """WasbRunner WSL-glue coverage (the LIVE detector path; previously only an ABC-
conformance
2     test existed while the parallel TrackNetRunner had ~21). WSL is mocked."""
3     import types
4     from unittest.mock import patch
5
6     from backend.pipeline.detectors.wasb_runner import WasbRunner, WasbConfig
7     from backend.pipeline.detectors.base import DetectorRunner
8
9
10    def _proc(rc=0, stdout="", stderr=""):
11        return types.SimpleNamespace(returncode=rc, stdout=stdout, stderr=stderr)
12
13
14    def test_is_a_detector_runner():
15        assert isinstance(WasbRunner(WasbConfig()), DetectorRunner)
16
17
18    def test_healthcheck_ok():
19        r = WasbRunner(WasbConfig())
20        with patch.object(r, "_wsl_bash", return_value=_proc(0, stdout="torch 2.6 cuda
True")):
21            ok, msg = r.healthcheck()
22            assert ok and "torch" in msg
23
24

```

```

25     def test_healthcheck_env_failure():
26         r = WasbRunner(WasbConfig())
27         with patch.object(r, "_wsl_bash", return_value=_proc(1, stderr="ModuleNotFoundError:
torch")):
28             ok, msg = r.healthcheck()
29             assert not ok and "failed" in msg.lower()
30
31
32     def test_healthcheck_no_wsl_binary():
33         r = WasbRunner(WasbConfig())
34         with patch.object(r, "_wsl_bash", side_effect=FileNotFoundError()):
35             ok, msg = r.healthcheck()
36             assert not ok and "wsl" in msg.lower()
37
38
39     def test_stage_video_success_and_failure():
40         r = WasbRunner(WasbConfig())
41         # Two WSL calls now: the readability preflight (echo READABLE) then the cp (echo
OK).
42         with patch.object(r, "_wsl_bash",
43                             side_effect=[_proc(0, stdout="READABLE"), _proc(0, stdout="OK")]):
44             assert r._stage_video(r"C:\v\clip.mp4", "clip.mp4") == "~/clips/clip.mp4"
45             # Source readable, but the cp itself fails -> None.
46             with patch.object(r, "_wsl_bash",
47                             side_effect=[_proc(0, stdout="READABLE"), _proc(1, stderr="disk
full")]):
48                 assert r._stage_video(r"C:\v\clip.mp4", "clip.mp4") is None
49
50
51     def test_run_predict_returns_none_when_infer_script_stage_fails(tmp_path):
52         r = WasbRunner(WasbConfig())
53         with patch.object(r, "_stage_infer_script", return_value=False):
54             assert r.run_predict(r"C:\v\clip.mp4", str(tmp_path), video_id="abc123abc123")
is None
55
56
57     # --- cache hygiene (keep_frames / min_free_gb / shared WSL helpers) -----
58
59     def test_from_indexing_cfg_reads_cleanup_knobs():
60         cfg = WasbConfig.from_indexing_cfg({"wasb": {"keep_frames": True, "min_free_gb":
25}})
61         assert cfg.keep_frames is True and cfg.min_free_gb == 25.0
62         default = WasbConfig.from_indexing_cfg({})
63         assert default.keep_frames is False and default.min_free_gb == 0.0
64
65
66     def test_wsl_rm_rf_builds_rm_command():
67         r = WasbRunner(WasbConfig())
68         with patch.object(r, "_wsl_bash", return_value=_proc(0)) as wb:
69             r._wsl_rm_rf("~/clips/a_frames", "~/clips/a.mp4")
70             inner = wb.call_args[0][0]
71             assert inner.startswith("rm -rf ") and "a_frames" in inner and "a.mp4" in inner
72
73
74     def test_wsl_rm_rf_noop_on_empty():
75         r = WasbRunner(WasbConfig())
76         with patch.object(r, "_wsl_bash") as wb:
77             r._wsl_rm_rf([])
78             wb.assert_not_called()
79
80
81     def test_wsl_free_gb_parses_bytes_else_none():
82         r = WasbRunner(WasbConfig())
83         with patch.object(r, "_wsl_bash", return_value=_proc(0, stdout=str(50 * 1024 ** 3) +
"\n")):

```

```

84         assert abs(r._wsl_free_gb("~/clips") - 50.0) < 0.01
85         with patch.object(r, "_wsl_bash", return_value=_proc(0, stdout="garbage")):
86             assert r._wsl_free_gb("~/clips") is None
87
88
89     def _run_predict_to_success(cfg, tmp_path, vid="abc123abc123"):
90         """Drive run_predict to its success branch with WSL fully mocked. Returns (rm_spy,
key)."""
91         r = WasbRunner(cfg)
92         key = f"clip__{vid[:12]}"
93         (tmp_path / f"{key}_wasb.csv").write_text("Frame,Visibility,X,Y\n") # success = CSV
exists
94         with patch.object(r, "_stage_infer_script", return_value=True), \
95             patch.object(r, "_stage_video", return_value=f "~/clips/{key}.mp4"), \
96             patch.object(r, "_wsl_bash", return_value=_proc(0)), \
97             patch.object(r, "_wsl_rm_rf") as rm:
98             out = r.run_predict(r"C:\v\clip.mp4", str(tmp_path), video_id=vid)
99         assert out is not None
100         return rm, key
101
102
103     def test_run_predict_cleans_frames_and_staged_video_on_success(tmp_path):
104         rm, key = _run_predict_to_success(WasbConfig(), tmp_path)
105         rm.assert_called_once()
106         purged = rm.call_args[0][0]
107         assert f "~/clips/{key}_frames" in purged and f "~/clips/{key}.mp4" in purged
108
109
110     def test_run_predict_keeps_frames_when_keep_frames_true(tmp_path):
111         rm, _ = _run_predict_to_success(WasbConfig(keep_frames=True), tmp_path)
112         rm.assert_not_called()
113

```

33. assistant (2026-06-16T02:25:46.242Z)

Now I understand the WSL/detector mocking pattern (patch `\_wsl\_bash`, `\_stage\_video`, `detector` injection). Let me look at how `fusion\_compare` consumes the per-video golden JSON (the load?transform?assert pattern), the `distill\_local` CSV-driven flow, and how the TrackNet/video is mocked. Let me also examine the existing trajectory CSV parsing and the gt_loader module.

34. user (2026-06-16T02:25:47.364Z)

```

1         """GPU-free tests for the fusion-P3 measurement (fusion_golden extractor +
fusion_compare LOVO).
2
3         The LOVO driver is exercised on a synthetic 6-video set where the SHUTTLE features are
4         non-separating (constant) but a PERSON feature cleanly separates rallies ? so the
harness must
5         report a positive fusion ?F1. The extractor is exercised with a stubbed detector (no
GPU).
6         """
7         import json
8         import os
9         from types import SimpleNamespace
10
11         import pytest
12
13         from backend.eval import fusion_compare
14         from backend.eval.fusion_compare import PERSON_FEATURE_NAMES, SHUTTLE_FEATURE_NAMES
15
16
17     def _write_video(d, name):
18         """A video whose shuttle features are identical for pos/neg (no signal) but whose
19         players_in_court separates rallies (4) from junk (0)."""
20         cands = []

```



```

21         for (s, e), pos in [((0.0, 10.0), 1), ((20.0, 30.0), 1), ((40.0, 41.0), 0), ((50.0,
51.0), 0)]:
22             shuttle = {fn: 1.0 for fn in SHUTTLE_FEATURE_NAMES}                # constant
-> non-separating
23             person = {fn: 0.0 for fn in PERSON_FEATURE_NAMES}
24             person["players_in_court"] = 4.0 if pos else 0.0                    # the
separating signal
25             person["in_court_ratio"] = 1.0 if pos else 0.0
26             cands.append({"start": s, "end": e, "shuttle": shuttle, "person": person,
"label": pos})
27             data = {"name": name, "fps": 30.0, "frame_width": 1920.0,
28                     "candidates": cands, "gts": [[0.0, 10.0], [20.0, 30.0]]}
29             with open(os.path.join(d, f"{name}_golden_features.json"), "w", encoding="utf-8") as
f:
30                 json.dump(data, f)
31
32
33     def test_fusion_compare_reports_person_driven_lift(tmp_path):
34         d = str(tmp_path)
35         for k in range(6):
36             _write_video(d, f"vid{k}")
37         videos = fusion_compare.load_videos(d)
38         assert len(videos) == 6
39         res = fusion_compare.compare(videos, iou=0.5)
40         assert res["num_videos"] == 6
41         assert res["variant_b"]["honest_lovo"] is True                          # LOVO-honest learned
variant
42         # Fusion (with the separating person feature) must beat shuttle-only (no signal).
43         assert res["delta"]["f1"] > 0.0
44         assert res["variant_b"]["aggregate"]["f1"] > res["variant_a"]["aggregate"]["f1"]
45         # format_result must not crash and must mention both variants.
46         out = fusion_compare.format_result(res, 0.5)
47         assert "shuttle-only" in out and "+person" in out
48
49
50     def test_fusion_compare_delta_stats_and_ci(tmp_path):
51         """delta_stats yields a paired bootstrap CI + sign test; format_result reports
them."""
52         d = str(tmp_path)
53         for k in range(6):
54             _write_video(d, f"vid{k}")
55         res = fusion_compare.compare(fusion_compare.load_videos(d), iou=0.5)
56         ds = fusion_compare.delta_stats(res)
57         assert set(ds) >= {"mean_delta", "ci_low", "ci_high", "sign_test",
"ci_excludes_zero"}
58         assert ds["ci_low"] - 1e-9 <= ds["mean_delta"] <= ds["ci_high"] + 1e-9    # CI
brackets mean (±fp)
59         assert ds["sign_test"]["wins"] == 6                                    # fusion wins on every video (person
feature separates)
60         assert ds["ci_excludes_zero"] is True
61         out = fusion_compare.format_result(res, 0.5)
62         assert "CI" in out and "sign test" in out
63
64
65     def test_fusion_compare_needs_two_videos(tmp_path):
66         _write_video(str(tmp_path), "only")
67         videos = fusion_compare.load_videos(str(tmp_path))
68         assert len(videos) == 1 # compare() / experiment.compare needs >=2 for honest LOVO
69
70
71     # ----- extractor assembly
72
73     def test_extract_video_assembles_shuttle_and_person(monkeypatch):
74         from backend.eval import fusion_golden
75         from backend.pipeline.detectors.tracknet_runner import TrackNetRunner

```

```

76
77     intervals = [(0.0, 10.0), (40.0, 41.0)]
78     x_shuttle = [[10.0, 0.3, 0.2, 0.5, 4.0], [1.0, 0.1, 0.05, 0.2, 0.0]]
79     points = [SimpleNamespace(frame=i, visible=True, x=500.0, y=(100.0 if i % 2 else
900.0))]
80         for i in range(10)]
81
82     monkeypatch.setattr(fusion_golden.distill_local, "build_candidates",
83                         lambda traj, fps, fw: (intervals, x_shuttle))
84     monkeypatch.setattr(TrackNetRunner, "parse_trajectory_csv", staticmethod(lambda
85     traj: points))
86
87     monkeypatch.setattr(fusion_golden, "load_golden_gts", lambda labels: [(0.0, 10.0)])
88
89     class _StubDetector:
90         def detections_over_windows(self, video, windows, frame_skip, progress_cb=None):
91             return [{"frames": {sf: [(1, (10.0, 10.0, 30.0, 40.0))]},
92                     "fps": 30.0, "frame_width": 1920.0, "frame_height": 1080.0}
93                     for (sf, ef) in windows]}
94
95     spec = {"name": "t", "trajectory": "bar.csv", "fps": 30.0, "frame_width": 1920.0,
96            "labels": "foo.rallies.csv"}
97     data = fusion_golden.extract_video(spec, detector=_StubDetector(), frame_skip=3,
98     iou=0.5)
99
100     assert data["name"] == "t"
101     assert data["gts"] == [(0.0, 10.0)]
102     assert len(data["candidates"]) == 2
103     c0 = data["candidates"][0]
104     assert set(c0["shuttle"]) == set(SHUTTLE_FEATURE_NAMES)
105     assert set(c0["person"]) == set(PERSON_FEATURE_NAMES)
106     assert c0["label"] == 1 # (0,10) matches the gt -> positive
107     assert data["candidates"][1]["label"] == 0 # (40,41) -> negative
108     assert all(isinstance(v, float) for v in c0["person"].values())
109
110     def test_fusion_golden_run_is_resumable(tmp_path, monkeypatch):
111         """run() skips a video whose JSON already exists (resume a long run); --fresh redoes
112         all."""
113         from backend.eval import fusion_golden
114         out_dir = str(tmp_path / "out")
115         os.makedirs(out_dir)
116         manifest = [
117             {"name": "alpha", "trajectory": "a.csv", "fps": 30.0, "frame_width": 1920.0,
118             "labels": "a.rallies.csv"},
119             {"name": "beta", "trajectory": "b.csv", "fps": 30.0, "frame_width": 1920.0,
120             "labels": "b.rallies.csv"},
121         ]
122         manifest_path = str(tmp_path / "m.json")
123         with open(manifest_path, "w", encoding="utf-8") as f:
124             json.dump(manifest, f)
125         # a prior run already produced alpha's JSON
126         with open(os.path.join(out_dir, "alpha_golden_features.json"), "w",
127         encoding="utf-8") as f:
128             json.dump({"name": "alpha", "fps": 30.0, "frame_width": 1920.0, "candidates":
129             [], "gts": []}, f)
130
131         extracted = []
132
133         def _stub(spec, **kw):
134             extracted.append(spec["name"])
135             return {"name": spec["name"], "fps": 30.0, "frame_width": 1920.0, "candidates":
136             [], "gts": []}
137
138         monkeypatch.setattr(fusion_golden, "extract_video", _stub)

```

```

132 fusion_golden.run(manifest_path, out_dir, fresh=False)
133 assert extracted == ["beta"] # alpha skipped (already extracted)
134
135 extracted.clear()
136 fusion_golden.run(manifest_path, out_dir, fresh=True)
137 assert extracted == ["alpha", "beta"] # --fresh redoes both
138
139
140 def test_fusion_golden_smallest_first(tmp_path, monkeypatch):
141     """--smallest-first orders by trajectory size; default keeps manifest order."""
142     from backend.eval import fusion_golden
143     out_dir = str(tmp_path / "out")
144     os.makedirs(out_dir)
145     (tmp_path / "big.csv").write_text("x" * 2000)
146     (tmp_path / "small.csv").write_text("x" * 20)
147     manifest = [
148         {"name": "big", "trajectory": str(tmp_path / "big.csv"), "fps": 30.0,
149 "frame_width": 1920.0, "labels": "b.rallies.csv"},
150         {"name": "small", "trajectory": str(tmp_path / "small.csv"), "fps": 30.0,
151 "frame_width": 1920.0, "labels": "s.rallies.csv"},
152     ]
153     mp = str(tmp_path / "m.json")
154     with open(mp, "w", encoding="utf-8") as f:
155         json.dump(manifest, f)
156
157     order = []
158
159     def _stub(spec, **kw):
160         order.append(spec["name"])
161         return {"name": spec["name"], "fps": 30.0, "frame_width": 1920.0, "candidates":
162 [], "gts": []}
163
164     monkeypatch.setattr(fusion_golden, "extract_video", _stub)
165
166     fusion_golden.run(mp, out_dir, smallest_first=True)
167     assert order == ["small", "big"] # smaller trajectory first
168
169     order.clear()
170     fusion_golden.run(mp, out_dir, smallest_first=False, fresh=True)
171     assert order == ["big", "small"] # manifest order preserved
172
173
174 def test_derive_video_path():
175     from backend.eval.fusion_golden import _derive_video_path
176     assert _derive_video_path({"labels": "X/Adarsh and Avi.rallies.csv"}) == "X/Adarsh
177 and Avi.MP4"
178     assert _derive_video_path({"video_path": "a.mp4", "labels": "b.rallies.csv"}) ==
179 "a.mp4"
180     with pytest.raises(ValueError):
181         _derive_video_path({"name": "n", "labels": "weird.csv"})
182

```

35. user (2026-06-16T02:25:47.815Z)

```

1     """Pure-CPU LOVO comparison: shuttle-only vs shuttle+person learned fusion on the golden
2 set.
3     The fusion-P3 measurement. Loads the per-video feature JSONs written by
4 `fusion_golden.py`
5 and runs the SHIPPED experiment harness (`experiment.compare`, honest leave-one-video-
6 out) ?
7     no new scoring/labeling math. The two variants differ ONLY by `feature_names` (the
8 harness's
9 designed seam): the person features are extra columns the learned LogisticRegression
10 weights,

```

```

7     so the held-out ?F1 is an honest answer to "does the person cue help?" ? not a hunch.
8
9     Run (no GPU):
10    python -m backend.eval.fusion_compare --features-dir output --record
11    """
12    from __future__ import annotations
13
14    import argparse
15    import glob
16    import json
17    import os
18    from typing import Any, Dict, List
19
20    from backend.eval import experiment
21    from backend.eval.eval_stats import paired_delta_ci
22    from backend.eval.fusion_golden import PERSON_FEATURE_NAMES, SHUTTLE_FEATURE_NAMES
23
24
25    def delta_stats(res: Dict[str, Any], metric: str = "f1") -> Dict[str, Any]:
26        """Paired (by video) B?A summary for one metric: mean delta + bootstrap CI + sign
test
27        (eval_stats, C1 ? "never a bare mean"). Aligns the two variants' per-video held-out
scores
28        by video name, so the headline ?F1 comes with how-wide + did-it-win-on-enough-
videos."""
29        by_a = {p["video"]: p[metric] for p in res["variant_a"]["per_video"]}
30        by_b = {p["video"]: p[metric] for p in res["variant_b"]["per_video"]}
31        vids = [p["video"] for p in res["variant_a"]["per_video"]]
32        return paired_delta_ci([by_a[v] for v in vids], [by_b[v] for v in vids])
33
34
35    def load_videos(features_dir: str) -> List[experiment.VideoCandidates]:
36        """Load every ``*_golden_features.json`` into a VideoCandidates (intervals + the 10
37        feature columns + golden gts). The harness derives labels from gts itself."""
38        videos: List[experiment.VideoCandidates] = []
39        for path in sorted(glob.glob(os.path.join(features_dir, "*_golden_features.json"))):
40            with open(path, encoding="utf-8") as f:
41                d = json.load(f)
42                cands = d["candidates"]
43                intervals = [(float(c["start"]), float(c["end"])) for c in cands]
44                features: Dict[str, List[float]] = {}
45                for fn in SHUTTLE_FEATURE_NAMES:
46                    features[fn] = [float(c["shuttle"][fn]) for c in cands]
47                for fn in PERSON_FEATURE_NAMES:
48                    features[fn] = [float(c["person"][fn]) for c in cands]
49                gts = [(float(a), float(b)) for a, b in d["gts"]]
50                videos.append(experiment.VideoCandidates(name=d["name"], intervals=intervals,
51                                                            features=features, gts=gts))
52        return videos
53
54
55    def compare(videos: List[experiment.VideoCandidates], iou: float = 0.5,
56               threshold: float = 0.5) -> Dict[str, Any]:
57        shuttle_only = experiment.Variant(name="shuttle_only",
feature_names=list(SHUTTLE_FEATURE_NAMES),
58               threshold=threshold)
59        fusion = experiment.Variant(name="shuttle_person_fusion",
60               feature_names=list(SHUTTLE_FEATURE_NAMES) +
list(PERSON_FEATURE_NAMES),
61               threshold=threshold)
62        return experiment.compare(videos, shuttle_only, fusion, iou=iou, honest=True)
63
64
65    def format_result(res: Dict[str, Any], iou: float) -> str:
66        a, b, d = res["variant_a"]["aggregate"], res["variant_b"]["aggregate"], res["delta"]

```

```

67         ds = delta_stats(res)
68         st = ds["sign_test"]
69         lines = [
70             f"=== Fusion P3 LOVO (n={res['num_videos']} golden videos, IoU>={iou}, honest)
===",
71             f"  shuttle-only : F1={a['f1']:.3f}  P={a['precision']:.3f}  R={a['recall']:.3f}
IoU={a['mean_iou']:.3f}",
72             f"  +person      : F1={b['f1']:.3f}  P={b['precision']:.3f}  R={b['recall']:.3f}
IoU={b['mean_iou']:.3f}",
73             f"  ?F1 = {d['f1']:.3f}  ({res['delta_pct']['f1']:.1f}%)
?P={d['precision']:.3f}  ?R={d['recall']:.3f}",
74             f"  paired ?F1 {ds['mean_delta']:.3f}  [95% CI {ds['ci_low']:.3f},
{ds['ci_high']:.3f}]  "
75             f"| sign test {int(st['wins'])}W/{int(st['losses'])}L (p={st['p_value']:.3f})  "
76             f"| CI clears 0: {ds['ci_excludes_zero']}",
77             "  per-video ?F1:",
78         ]
79         for pv in res["per_video_delta"]:
80             lines.append(f"    {pv['video']:<26s} ?F1={pv['f1']:.3f}")
81         return "\n".join(lines)
82
83
84     def main() -> None:
85         ap = argparse.ArgumentParser(description="LOVO fusion lift: shuttle-only vs
shuttle+person (golden set).")
86         ap.add_argument("--features-dir", default="output")
87         ap.add_argument("--iou", type=float, default=0.5)
88         ap.add_argument("--record", action="store_true", help="append the result to the
experiment leaderboard")
89         args = ap.parse_args()
90
91         videos = load_videos(args.features_dir)
92         if len(videos) < 2:
93             raise SystemExit(f"need >=2 *_golden_features.json in {args.features_dir} (found
{len(videos)})")
94         res = compare(videos, iou=args.iou)
95         print(format_result(res, args.iou))
96         if args.record:
97             row = experiment.record_experiment(res)
98             print(f"recorded -> eval_baselines/experiment_leaderboard.json
({row['timestamp']}")
99
100
101     if __name__ == "__main__":
102         main()
103

```

36. assistant (2026-06-16T02:25:52.475Z)

This is the canonical "load committed artifact ? run transform ? assert" pattern, but currently fixtures are written inline to tmp_path, not committed. Let me check the gt_loader module, the TrackNet runner's CSV parsing (the trajectory format), the regression/per-user regression modules, and look at where output/ golden artifacts live and whether anything is gitignored. Let me also check the .gitignore and the CODE_MAP for file-placement conventions.

37. user (2026-06-16T02:25:53.895Z)

```

1     """Canonical ground-truth interval loader ? THE one GT/golden-CSV reader (ADR-008
prereq).
2
3     The repo historically carried two incompatible loaders:
4     - ``annotations.load_annotations``: positional ``start,end`` (the collector's
5       ``curate export`` format), raises on bad rows;
6     - ``calibrate_wasb.load_golden_gts``: DictReader requiring ``start_time/end_time``
7       (the rally-annotator golden format), silently skipping bad rows ? fed 7 modules.

```

```

8
9     A file in one convention fed to the other loader yielded zero/garbage intervals with no
10    error. This module accepts BOTH conventions (header-mapped, else positional), so every
11    quantity the ship-gate depends on (gt_hash, LOVO scores, the future consent fingerprint)
12    flows through one reader. Both legacy entry points now delegate here.
13
14    Accepted shapes (one rally per row; blank lines and ``#`` comments ignored):
15    - header with ``start``/``end`` OR ``start_time``/``end_time`` columns (any other
16      columns ? rally_number, ending_reason, sport, ? ? are ignored);
17    - no header: columns 0,1 positionally.
18    Timestamps: decimal seconds or ``MM:SS`` / ``HH:MM:SS``
19    (``annotations.parse_timestamp``).
20    """
21    import csv
22    import logging
23    from typing import List, Optional, Tuple
24
25    from backend.eval.annotations import parse_timestamp
26
27    logger = logging.getLogger(__name__)
28
29    Interval = Tuple[float, float]
30
31    _START_NAMES = ("start", "start_time")
32    _END_NAMES = ("end", "end_time")
33
34    def _header_columns(row: List[str]) -> Optional[Tuple[int, int]]:
35        """If `row` is a header naming start/end columns, return their indices, else
36        None."""
37        cells = [(c or "").strip().lower() for c in row]
38        start_idx = next((i for i, c in enumerate(cells) if c in _START_NAMES), None)
39        end_idx = next((i for i, c in enumerate(cells) if c in _END_NAMES), None)
40        if start_idx is not None and end_idx is not None:
41            return start_idx, end_idx
42        return None
43
44    def load_gt_intervals(csv_path: str, *, strict: bool = True) -> List[Interval]:
45        """Load GT rally intervals from either golden-CSV convention.
46
47        strict=True (corpus/gate use): raise ValueError with file:line on any malformed row
48        (bad timestamp, end <= start, negative start) so labeling mistakes fail loudly.
49        strict=False (legacy tuning-driver behavior): skip bad rows, but LOG how many were
50        skipped ? a silently shrinking GT set flatters precision.
51
52        Returns intervals sorted by start; overlapping GT intervals are logged as a warning
53        (they break one-to-one matching and usually indicate a labeling error).
54        """
55        intervals: List[Interval] = []
56        skipped = 0
57        cols: Optional[Tuple[int, int]] = None # (start_idx, end_idx); None until detected
58        saw_header = False
59
60        with open(csv_path, newline="", encoding="utf-8-sig") as f:
61            for lineno, row in enumerate(csv.reader(f), start=1):
62                if not row or not (row[0] or "").strip() or row[0].strip().startswith("#"):
63                    continue
64                if cols is None and not saw_header:
65                    header = _header_columns(row)
66                    if header is not None:
67                        cols = header
68                        saw_header = True
69                    continue # consume the header row
70                cols = (0, 1) # headerless: positional start,end

```

```

71         assert cols is not None      # every branch above assigns it
72         si, ei = cols
73         try:
74             if len(row) <= max(si, ei):
75                 raise ValueError(f"expected at least {max(si, ei) + 1} columns, got
{row!r}")
76                 start = parse_timestamp(row[si])
77                 end = parse_timestamp(row[ei])
78                 if start < 0:
79                     raise ValueError(f"negative start ({start})")
80                 if start >= end:
81                     raise ValueError(f"start ({start}) >= end ({end})")
82         except ValueError as e:
83             if strict:
84                 raise ValueError(f"{csv_path}:{lineno}: {e}") from e
85             skipped += 1
86             continue
87         intervals.append((start, end))
88
89     if skipped:
90         logger.warning("%s: skipped %d malformed GT row(s) (strict=False)", csv_path,
skipped)
91
92     intervals.sort(key=lambda iv: iv[0])
93     for (s1, e1), (s2, e2) in zip(intervals, intervals[1:]):
94         if s2 < e1:
95             logger.warning("%s: overlapping GT intervals (%.3f-%.3f vs %.3f-%.3f) ? "
96                             "labeling error? Overlaps break one-to-one matching.",
97                             csv_path, s1, e1, s2, e2)
98     return intervals
99

```

38. user (2026-06-16T02:25:54.367Z)

```

1     # Byte-compiled / optimized / DLL files
2     __pycache__/
3     *.py[cod]
4     *$py.class
5     *.pt
6
7     # Local environment secrets
8     .env
9     *.env
10    .env.*
11
12    # C extensions
13    *.so
14
15    # Distribution / packaging
16    .Python
17    build/
18    develop-eggs/
19    dist/
20    downloads/
21    eggs/
22    .eggs/
23    lib/
24    lib64/
25    parts/
26    sdist/
27    var/
28    wheels/
29    share/python-wheels/
30    *.egg-info/
31    .installed.cfg

```

```

32     *.egg
33     MANIFEST
34
35     # SQLite databases & Local output files
36     *.db
37     *.db-journal
38     *.db-wal
39     output/
40     output_dir/
41     backend/static/highlights_*.mp4
42     backend/static/*_compiled.mp4
43
44     # OS generated files
45     .DS_Store
46     Thumbs.db
47     desktop.ini
48
49     # Coverage data
50     .coverage
51     .coverage.*
52     htmlcov/
53     .pytest_cache/
54
55     # Local Claude Code config (machine-specific)
56     .claude/
57     .clocovleerlaage
58     █
59     █
60     # Throwaway repro scripts + PR-body / handoff scratch notes (already excluded from
61     # ruff + pytest; never meant to be committed ? keep them out of `git status` too).
62     scratch/
63     .venv/
64

```

39. user (2026-06-16T02:25:54.554Z)

```

1     """GPU-free tests for the per-user NO-REGRESSION guard
(backend.eval.per_user_regression).
2
3     Mirrors the synthetic-`VideoCandidates` pattern in test_fusion_compare.py /
test_per_user_eval.py:
4     each video carries 2 positive candidates (match the gts) + 2 negative candidates (junk),
with a
5     ``players_in_court`` feature that separates rallies (4) from junk (0). Two *static*
variants then
6     make the decision deterministic and bootstrap-free in expectation:
7
8     - a **GOOD** variant gates on ``min_players>=1`` ? keeps only the rallies ? perfect
F1;
9     - an **OVERFIRE** variant has no gate ? keeps every candidate ? low F1 (false
positives).
10
11     Swapping OVERFIRE?GOOD is a clear improvement (ACCEPT); GOOD?OVERFIRE is a clear
regression
12     (REJECT, keep incumbent); GOOD?GOOD is a tie (keep incumbent). Pure-CPU, deterministic.
13     """
14     import pytest
15
16     from backend.eval.experiment import ExperimentError, Variant, VideoCandidates
17     from backend.eval.per_user_regression import (
18         PerUserBaseline,
19         PerUserRegressionResult,
20         baseline_key,
21         evaluate_no_regression,
22         load_baselines,

```



```

23         save_baseline,
24     )
25
26     PLAYERS = "players_in_court"
27
28
29     def _video(name: str) -> VideoCandidates:
30         """A 4-candidate video: 2 rallies (players=4, match the gts) + 2 junk windows
(players=0)."""
31         intervals = [(0.0, 10.0), (20.0, 30.0), (40.0, 41.0), (50.0, 51.0)]
32         players = [4.0, 4.0, 0.0, 0.0]
33         gts = [(0.0, 10.0), (20.0, 30.0)]
34         return VideoCandidates(name=name, intervals=intervals, features={PLAYERS: players},
gts=gts)
35
36
37     def _good() -> Variant:
38         """Gates on players>=1 ? keeps only the two rally windows ? perfect held-out F1."""
39         return Variant(name="good", min_players=1)
40
41
42     def _overfire() -> Variant:
43         """No gate ? keeps every candidate (2 rallies + 2 junk) ? low precision, low F1."""
44         return Variant(name="overfire")
45
46
47     def _corpus(n: int = 6):
48         return [_video(f"v{i}") for i in range(n)]
49
50
51     def test_accepts_clear_improvement():
52         """Candidate (gated, perfect F1) clearly beats the over-firing incumbent ? ACCEPT
the swap."""
53         res = evaluate_no_regression(_corpus(6), control=_overfire(), candidate=_good(),
seed=0)
54         assert isinstance(res, PerUserRegressionResult)
55         assert res.accept is True
56         assert res.keep_incumbent is False
57         assert res.regressed is False
58         assert res.candidate_f1 > res.control_f1
59         assert res.mean_delta > 0.0
60         assert res.ci_excludes_zero is True
61         assert res.ci_low > 0.0 # CI clears 0 on the RIGHT side
62         assert res.sign_test["wins"] > res.sign_test["losses"]
63         assert "accept" in res.reason.lower()
64
65
66     def test_rejects_clear_regression_keeps_incumbent():
67         """Candidate (over-firing) is clearly worse than the gated incumbent ? REJECT, keep
incumbent."""
68         res = evaluate_no_regression(_corpus(6), control=_good(), candidate=_overfire(),
seed=0)
69         assert res.accept is False
70         assert res.keep_incumbent is True # the user keeps their better
incumbent
71         assert res.regressed is True
72         assert res.candidate_f1 < res.control_f1
73         assert res.mean_delta < 0.0
74         assert res.ci_excludes_zero is True
75         assert res.ci_high < 0.0 # whole CI below 0 ? a real
regression
76         assert "regression" in res.reason.lower()
77
78
79     def test_tie_insufficient_evidence_keeps_incumbent():

```

```

80         """Identical variants ? F1 is exactly 0, CI spans 0 ? keep the incumbent (never
swap on noise)."""
81         res = evaluate_no_regression(_corpus(6), control=_good(), candidate=_good(), seed=0)
82         assert res.accept is False
83         assert res.keep_incumbent is True
84         assert res.regressed is False          # a tie is NOT a regression ...
85         assert res.mean_delta == pytest.approx(0.0) # ... it is no measurable difference
86         assert res.ci_excludes_zero is False
87         assert res.candidate_f1 == pytest.approx(res.control_f1)
88         assert "insufficient evidence" in res.reason.lower()
89
90
91     def test_no_videos_keeps_incumbent():
92         """An empty corpus ? no evidence ? keep the incumbent, never a crash."""
93         res = evaluate_no_regression([], control=_good(), candidate=_overfire(), seed=0)
94         assert res.n == 0
95         assert res.accept is False
96         assert res.keep_incumbent is True
97         assert res.regressed is False
98
99
100    def test_unlabeled_video_raises():
101        """A held-out video with no golden labels can't be scored ? ExperimentError (from
evaluate_variant)."""
102        videos = [_video("v0"),
103                  VideoCandidates(name="unlabeled", intervals=[(0.0, 1.0)],
104                                features={PLAYERS: [1.0]}, gts=None)]
105        with pytest.raises(ExperimentError):
106            evaluate_no_regression(videos, control=_good(), candidate=_overfire(), seed=0)
107
108
109    def test_result_as_dict_round_trips_fields():
110        """as_dict() exposes the full structured result for telemetry / audit logging."""
111        res = evaluate_no_regression(_corpus(6), control=_overfire(), candidate=_good(),
seed=0)
112        out = res.as_dict()
113        for key in (
114            "accept", "keep_incumbent", "reason", "n", "control_f1", "candidate_f1",
115            "mean_delta", "ci_low", "ci_high", "ci_excludes_zero", "regressed", "sign_test",
116        ):
117            assert key in out
118        assert out["accept"] is True and out["keep_incumbent"] is False
119
120
121    # ----- baseline record
round-trip
122
123    def test_baseline_key_null_user_is_local():
124        """A NULL user_id (local single-user) keys as the literal 'local'; a real id is
namespaced."""
125        assert baseline_key(None, "final") == "local::final"
126        assert baseline_key("u123", "final") == "u123::final"
127        assert baseline_key("u123", "shadow") == "u123::shadow"
128
129
130    def test_baseline_record_round_trips_through_save_load(tmp_path):
131        """A user_id::stage baseline record survives save ? load unchanged (sibling to
regression_baseline.json)."""
132        path = str(tmp_path / "per_user_baseline.json")
133        rec = PerUserBaseline(user_id="u123", stage="final", control_f1=0.875,
134                              control_spec_hash="abc123def456", n=6, iou=0.5)
135        key = save_baseline(rec, path=path)
136        assert key == "u123::final"
137
138        loaded = load_baselines(path)

```

```

139     assert key in loaded
140     assert loaded[key] == rec.as_dict()
141     assert loaded[key]["control_f1"] == pytest.approx(0.875)
142     assert loaded[key]["user_id"] == "u123"
143
144     # A second user/stage coexists under its own key (read-modify-write, not clobber).
145     rec2 = PerUserBaseline(user_id=None, stage="shadow", control_f1=0.5,
146                           control_spec_hash="000", n=3, iou=0.5)
147     save_baseline(rec2, path=path)
148     both = load_baselines(path)
149     assert set(both) == {"u123::final", "local::shadow"}
150     assert both["local::shadow"]["user_id"] is None
151
152
153     def test_evaluate_can_snapshot_incumbent_baseline(tmp_path):
154         """Passing baseline_path snapshots the incumbent's measured numbers without changing
155         the decision."""
156         path = str(tmp_path / "pu.json")
157         res = evaluate_no_regression(_corpus(6), control=good(), candidate=overfire(),
158                                   user_id="u9", stage="final", baseline_path=path,
159                                   seed=0)
160         # decision is unaffected by the snapshot (this is still a regression -> keep
161         incumbent)
162         assert res.regressed is True and res.keep_incumbent is True
163         saved = load_baselines(path)
164         key = baseline_key("u9", "final")
165         assert key in saved
166         # the snapshot records the INCUMBENT (control) numbers, not the candidate's
167         assert saved[key]["control_f1"] == pytest.approx(res.control_f1)
168         assert saved[key]["n"] == res.n

```

40. assistant (2026-06-16T02:25:57.963Z)

Now I have a thorough picture. Let me look at how the TrackNet trajectory CSV format is parsed (the per-frame format), check the served_gate_a regression test (an existing per-video regression pattern), and read the CODE_MAP section on file placement to nail down where new fixtures should go.

41. user (2026-06-16T02:25:58.929Z)

```

1     """SERVED Gate-A regression litmus ? the *production* FusionSegmenter path on real
2     footage.
3
4     The flagged gap (#146): the served default flip (`default_segmenter=fusion_hybrid` +
5     `min_crossings=2`) is **mock-tested only** on real video (`test_fusion_hybrid.py`
6     drives a
7     MagicMock runner). The offline ablation litmus
8     (`test_ablation.test_litmus_reproduces_gate_a`)
9     validates the Gate-A *signal* ? but on the **permissive proposal windowing**
10    (`distill_local.PROPOSAL`), NOT the production operating point. This module closes the
11    gap: it
12    runs the real ``FusionSegmenter`` decision seams (`_enrich_candidate_stats` net-
13    crossings +
14    `_select_for_handoff` Gate A) over windows from the **production**
15    `trajectory_to_action_windows` config, on the 6 cached golden WASB trajectories, and
16    asserts
17    the served path's measured behaviour.
18
19    GPU-free / Gemini-free (person cue OFF) ? it stops at the AI-handoff boundary. Skips
20    cleanly in
21    CI where the golden trajectories are absent; runs on the owner box where they are
22    present.
23
24    HONEST FINDING baked into the assertions (2026-06-14): on the **production windowing**

```

```

Gate A is
17     NOT the +0.074 win the offline harness reports. Production windowing is already
conservative
18     (velocity_threshold 0.02, merge_gap 2.0, min_window 2.0) ? few long windows, over-seg
ratio
19     already < 1 ? there is little held-shuttle junk left for Gate A to cut. So the served
?F1 is
20     ~neutral (slightly negative), NOT +0.074. The +0.074 reproduces ONLY under the
permissive
21     proposal windowing (asserted in ``test_proposal_windowing_reproduces_offline_lift``
below ? that
22     is what the offline litmus measures). These tests pin BOTH facts so a future windowing
change
23     that silently revives ? or breaks ? either one trips the litmus.
24     ""
25     from __future__ import annotations
26
27     import json
28     import os
29
30     import pytest
31
32     # cv2 is imported transitively by trajectory_hybrid (the served engine). On a box
without it
33     # (CI), skip the whole module rather than erroring at import.
34     pytest.importorskip("cv2")
35
36     from backend.eval import served_gate_a as sga # noqa: E402
37     from backend.pipeline.detectors.tracknet_runner import TrackNetRunner # noqa: E402
38     from backend.pipeline.detectors import rally_gate # noqa: E402
39     from backend.eval.calibrate_wasb import load_golden_gts # noqa: E402
40     from backend.eval.metrics import score # noqa: E402
41     from backend.eval import segmentation_metrics as _segm # noqa: E402
42
43     # The golden manifest lives in the MAIN checkout's output/ (gitignored; absent in a
fresh
44     # worktree / CI). Resolve it relative to this repo root first, with an env override for
an
45     # out-of-tree checkout.
46     _REPO_ROOT = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
47     MANIFEST = os.environ.get(
48         "SERVED_GATE_A_MANIFEST",
49         os.path.join(_REPO_ROOT, "output", "human_lovo_manifest.json"),
50     )
51
52
53     def _golden_specs():
54         """Return the manifest specs whose trajectory + label files are all present, else
[], """
55         if not os.path.isfile(MANIFEST):
56             return []
57         with open(MANIFEST, encoding="utf-8") as f:
58             specs = json.load(f)
59         ok = [s for s in specs
60              if os.path.isfile(str(s.get("trajectory", "")))
61              and os.path.isfile(str(s.get("labels", "")))]
62         return ok if len(ok) >= 6 else []
63
64
65     _SPECS = _golden_specs()
66     _skip = pytest.mark.skipif(
67         not _SPECS,
68         reason=f"golden trajectories absent (manifest {MANIFEST!r}); runs on the owner box",
69     )
70

```

```

71
72 # ----- served-path
litmus
73
74 @_skip
75 def test_served_path_runs_on_all_six_golden_videos():
76     """The production FusionSegmenter decision path runs end-to-end (no GPU/Gemini) on
all 6
77     golden trajectories and yields a handoff window list for each, for both gate
settings."""
78     results = sga.evaluate_manifest(_SPECS)
79     assert len(results) == 6
80     for r in results:
81         assert r.num_gt > 0
82         assert r.n_handoff_off >= r.n_handoff_on # Gate A only ever drops windows,
never adds
83
84
85 @_skip
86 def test_served_gate_a_does_not_regress_f1():
87     """SERVED no-regression guard: on the production windowing Gate A must not
MATERIALLY hurt
88     F1. It is ~neutral here (the held-shuttle junk the offline harness sees is already
merged
89     away by the conservative production windowing) ? so we assert the mean ?F1 sits in a
tight
90     band around 0, never a real regression. (The +0.074 lift lives under the permissive
proposal
91     windowing ? see the dedicated test below.)"""
92     results = sga.evaluate_manifest(_SPECS)
93     agg = sga.aggregate(results)
94     # Honest measured value (2026-06-14): mean served ?F1 ? -0.008. Pin a tolerance band
so a
95     # real regression (Gate A actively destroying served F1) trips, while the measured
~neutral
96     # result passes.
97     assert -0.03 <= agg["mean_delta_f1"] <= 0.03, agg
98     # And Gate A must never collapse served F1 to ~0 on the production path.
99     assert agg["mean_f1_on"] >= 0.5 * agg["mean_f1_off"] - 1e-9, agg
100
101
102 @_skip
103 def test_served_gate_a_does_not_increase_over_segmentation():
104     """Gate A only ever DROPS windows, so the served over-seg ratio must not rise when
it's on
105     (the direction-of-effect invariant ? even where the absolute lift is ~neutral)."""
106     results = sga.evaluate_manifest(_SPECS)
107     agg = sga.aggregate(results)
108     assert agg["mean_seg_ratio_on"] <= agg["mean_seg_ratio_off"] + 1e-9, agg
109     # Per-video too: the gate is monotone (kept windows ? all windows).
110     for r in results:
111         assert r.seg_ratio_on <= r.seg_ratio_off + 1e-9, r
112
113
114 @_skip
115 def test_served_uses_production_windowing_operating_point():
116     """Pin the served operating point to config.json's indexing defaults ? if someone
retunes
117     production windowing, this litmus's premise (and the divergence it documents) is
revisited
118     deliberately, not silently."""
119     assert sga.PROD_VELOCITY_THRESH == 0.02
120     assert sga.PROD_MERGE_GAP == 2.0
121     assert sga.PROD_MIN_WINDOW == 2.0
122

```

```

123
124     # ----- the +0.074 lives under PROPOSAL
windowing
125
126     # distill_local.PROPOSAL ? the permissive, recall-oriented candidate windowing the
OFFLINE
127     # ablation/fusion_compare harness builds on (low velocity thresh, short merge/min-dur ?
many
128     # small windows). The +0.074 Gate-A lift is a property of THIS windowing, not the served
one.
129     _PROPOSAL = (0.005, 1.0, 0.5)
130
131
132     @_skip
133     def test_proposal_windowing_reproduces_offline_lift():
134         """CONTROL / root-cause pin: under the OFFLINE harness's permissive proposal
windowing, the
135         SAME ``rally_gate`` net-crossing gate the served seam uses reproduces the offline
litmus
136         (?F1 ? +0.074, 6/6 wins, over-seg 1.68?1.01). This proves the served ~neutral result
is a
137         WINDOWING difference, not a broken gate ? and locks the offline number to its true
premise."""
138         d_on, d_off, seg_on, seg_off = [], [], [], []
139         for s in _SPECS:
140             fps, fw = float(s["fps"]), float(s["frame_width"])
141             pts = TrackNetRunner.parse_trajectory_csv(s["trajectory"])
142             wins = TrackNetRunner.trajectory_to_action_windows(
143                 pts, fps=fps, frame_width=fw, velocity_thresh=_PROPOSAL[0],
144                 merge_gap=_PROPOSAL[1], min_window_duration=_PROPOSAL[2])
145             iv = [(w["start"], w["end"]) for w in wins]
146             est = rally_gate.estimate_play_axis(pts)
147             gated = rally_gate.gate_windows(pts, iv, fps, min_crossings=0, estimate=est)
148             on = [(g.start, g.end) for g in gated if g.crossings >= 2]
149             gts = load_golden_gts(s["labels"])
150             d_on.append(score(on, gts).f1)
151             d_off.append(score(iv, gts).f1)
152             seg_on.append(_segm.segment_count_ratio(on, gts))
153             seg_off.append(_segm.segment_count_ratio(iv, gts))
154         n = len(d_on)
155         mean_delta = (sum(d_on) - sum(d_off)) / n
156         wins = sum(1 for a, b in zip(d_on, d_off) if a > b + 1e-9)
157         assert mean_delta == pytest.approx(0.074, abs=0.01), mean_delta
158         assert wins == 6, wins
159         assert sum(seg_off) / n == pytest.approx(1.68, abs=0.05)
160         assert sum(seg_on) / n == pytest.approx(1.01, abs=0.05)
161

```

42. user (2026-06-16T02:25:59.907Z)

```

237:     def parse_trajectory_csv(csv_win_path: str) -> List[TrajectoryPoint]:
238-         """Parse a TrackNet predict CSV (columns: Frame,Visibility,X,Y)."""
239-         points: List[TrajectoryPoint] = []
240-         with open(csv_win_path, newline="") as f:
241-             reader = csv.DictReader(f)
242-             for row in reader:
243-                 try:
244-                     points.append(TrajectoryPoint(
245-                         frame=int(row["Frame"]),
246-                         visible=int(row["Visibility"]) == 1,
247-                         x=float(row["X"]),
248-                         y=float(row["Y"]),
249-                     ))
250-                 except (KeyError, ValueError):
251-                     continue

```

```

252-         points.sort(key=lambda p: p.frame)
253-         return points
254-
255-     @staticmethod
256-     def trajectory_to_action_windows(
257-         points: List[TrajectoryPoint],
258-         fps: float,
259-         frame_width: float,
260-         chunk_start: float = 0.0,
261-         velocity_thresh: float = 0.02,
262-         merge_gap: float = 2.0,
263-         min_window_duration: float = 2.0,
264-         chunk_index: Optional[int] = None,
265-     ) -> List[Dict[str, Any]]:
266-         """Convert a shuttle trajectory into candidate rally windows.
267-
268-         A frame is "active" when the shuttle is visible AND its inter-frame
269-         displacement (normalised by frame width, per second) exceeds
270-         ``velocity_thresh`` ? i.e. the shuttle is in flight. Active frames within
271-         ``merge_gap`` seconds of each other are grouped into one window; short
272-         windows are dropped. Mirrors HybridYoloSegmenter's windowing so the dict

```

43. assistant (2026-06-16T02:26:04.296Z)

I now have everything I need. The `test_served_gate_a_regression.py` is the key existing per-video regression pattern ? it reads a manifest from `output/` (gitignored), skips when absent, and runs the served path over real golden trajectories. This is exactly the model for new auto-discovered per-video fixtures. Let me do a couple final confirmations: check the CODE_MAP file-placement section and confirm there's truly no committed test-fixture directory, plus check what `output/` currently holds and whether artifacts/ (untracked) has anything relevant.

44. user (2026-06-16T02:26:06.749Z)

```

7:[Omitted long matching line]
32:| A test | `tests/test_*.py` | pytest auto-discovers; the full-suite lane has a **70%
coverage floor**. Pure/offline + mocked (no GPU/WSL/network). |
36:| Eval baseline / leaderboard JSON | `eval_baselines/` | Tracked (survives a clean checkout);
the no-regression gate reads it. |
42:(grandfathered ? imported by `tests/test_eval_harness.py`). The old diagnostic `setup.py`
moved to
92:GT CSV -> annotations.load_annotations / calibrate_wasb.load_golden_gts -> List[Interval]
141:[Omitted long matching line]
142:[Omitted long matching line]
158:[Omitted long matching line]
165:- `@backend/eval/calibrate_wasb.py` ? WASB tuning CLI. `::BASELINE=(0.02,2.0,2.0)`,
`::default_grid` (45 cfgs), `::make_predict_fn` (parses trajectory once, closes over
`trajectory_to_action_windows`), `::load_golden_gts` ($`start_time,end_time` cols; ? SILENTLY
skips bad rows ? opposite of annotations.load_annotations). ? `{load_golden_gts,
make_predict_fn, BASELINE}` imported by 4 downstream drivers (calibrate_local,
export_wasb_segments, gemini_refine, audio/e1_probe).
167:- `@backend/eval/export_wasb_segments.py` ? tuned cfg -> golden/slicer CSV + comparison +
optional clip cut. `::TUNED=(0.05,1.0,1.0)` (? from ONE video), `::SEG_FIELDS`, `::COMP_FIELDS`,
`::build_comparison`, `::seconds_to_mmss`, `::maybe_slice` (lazy-imports rally_slicer).
`--slice` requires `--video`. ? `write_segments_csv` output loads straight into
`rally_slicer.load_rally_labels` AND re-imported by gemini_refine ? schema is load-bearing.
178:[Omitted long matching line]
179:[Omitted long matching line]
183:- `@backend/eval/audio/e1_probe.py` ? E1 GO/NO-GO diagnostic (no training/fusion).
`::cluster_hits` (groups thwacks; ? `min_hits>=2` so a lone service-feed hit isn't a rally),
`::run` (discrimination ratio + recall recovery + audio-only P/R/F1), `::main`. reuses
`hit_detector` + `calibrate_wasb.{load_golden_gts,make_predict_fn,BASELINE}` + `metrics`.
187:- `@backend/tools/rally_slicer.py` ? standalone CLI. `::compute_clip_windows` (pure, unit-
tested), `::load_rally_labels` (golden schema; ? silently skips degenerate rows),
`::slice_rallies`, `::ClipWindow` (rally:str), `::BEGIN_PADDING=0.5`/`::END_PADDING=1.0` (?
duplicate compiler's by copy). `read_time` -> backend/eval/annotations.parse_timestamp`.

```

```

216:[Omitted long matching line]
242:$ **GT / golden CSVs** (TWO conventions ? do not conflate):
244:- golden/slicer (`calibrate_wasb.load_golden_gts`, `rally_slicer.load_rally_labels`,
`export.SEG_FIELDS`): `rally_number,start_time,end_time[,duration,start_mmss,end_mmss]`;
SILENTLY skips bad rows.
273:- Collector also emits golden rally labels (`rally_number,start_time,end_time,...`) ->
consumed by `@backend/eval/calibrate_wasb.py` / `export_wasb_segments.py` /
`@backend/tools/rally_slicer.py`.
295:- **`calibrate_wasb.{load_golden_gts, make_predict_fn, BASELINE}` is a de-facto shared API**
for calibrate_local, export_wasb_segments, gemini_refine, e1_probe ? changing its golden-CSV
schema or `BASELINE` ripples widely. `gemini_refine.merge_overlaps` reused by `distill_local`;
`harness.get_predictions` reused by `regression.regress`.
327:| Run the whole test suite | `@run_all_tests.py` (repo root); or `pytest tests/` directly |
336:## 8. TEST MAP (`tests/test_*.py` -> subsystem)
356:| `test_calibrate_wasb.py` | `eval.calibrate_wasb` load_golden_gts/make_predict_fn/grid/run,
BASELINE |

```

45. user (2026-06-16T02:26:09.088Z)

```

=== output/ (gitignored) ===
GX010125_highlights.mp4
GX010125_highlights_min5.mp4
GX010125_run
KushagraAviHighlight-Trailor.mp4
MahadevpuraSingles_highlights.mp4
ab_experiment_n6.json
adarsh_avi_full_vs_human.csv
adarsh_avi_singles_golden_features.json
adarsh_avi_vs_human.csv
adarsh_baseline_gated.csv
adarsh_baseline_segments.csv
adarsh_gemini_fullvideo.csv
adarsh_gemini_refined.csv
adarsh_gemini_smoke.csv
adarsh_min5s_segments.csv
adarsh_tuned_segments.csv
badminton_indexer.db
chunks
e2e_reel.mp4
e2e_server_log.txt
fusion_golden.log
gateA_served_verify
gbaaddy_golden_features.json
gbaaddy_vs_human.csv
gx010125_min5_stitch.log
gx010125_run.log
gx010125_transcode.log
highlights_work
human_lovo_manifest.json
kushagra_1280.mp4
kushagra_1280.report.json
kushagra_1280.report.md
kushagra_1280.summary.md
kushagra_highlights_work
kushagra_singles_golden_features.json
kushagra_vs_human.csv
largetest_doubles_golden_features.json
largetestvideo_vs_human.csv
local_calib_manifest.json
local_compare.db
=== artifacts/ (untracked) ===
artifacts/:
rally_tal_gen0

artifacts/rally_tal_gen0:

```


0.1.0

```
artifacts/rally_tal_gen0/0.1.0:
manifest.json
weights.json
=== any committed *_golden_features.json or manifest? ===
backend/pipeline/segmenters/trajectory_hybrid.py
tests/test_trajectory_hybrid_equivalence.py
=== git tracked files under output ===
```

46. assistant (2026-06-16T02:26:12.745Z)

I now have a complete picture. Let me read the CODE_MAP \$0 file-placement table and the TEST MAP section to confirm exactly where new fixtures should go, plus check the line 7 / 32 omitted context.

47. user (2026-06-16T02:26:13.885Z)

```
20      ## 0. REPO LAYOUT ? where new files go (READ BEFORE committing a new file)
21
22      Top-level tree and the **placement rule** for each kind of file. When unsure, prefer an
existing
23      **registry / extension point** (`?`) over a new top-level file.
24
25      | You're adding? | Put it in? | Notes |
26      |---|---|---|
27      | Backend / runtime code | `backend/<subsystem>/` |
`pipeline/{segmenters,detectors,validators}`, `providers/`, `sports/`, `annotations/`,
`infrastructure/`, `reporting/`, `stitcher/`, `utils/`, `config/`, `features/`, `api/`. Most
additions are **register a class in the relevant `?` registry**, not a new module tree. |
28      | **Eval LIBRARY** (importable, pure ? metrics, calibration, a scorer, a feature) |
`backend/eval/` | Pure core, no side-effects at import; CLI/I-O lives in a separate driver.
These are **imported widely** (e.g. `calibrate_wasb` by 8 sites) ? they are library, not
scripts; do NOT move them out. |
29      | **Standalone run-driver / one-off entry CLI** (has `__main__`, NOT imported by app
code) | **`scripts/`** | e.g. `scripts/run_phase3_eval.py`, `scripts/run_process_and_stitch.py`.
Run as `python scripts/<name>.py`. If it needs `load_dotenv()`/`sys.path` before importing
backend, add `scripts/<name>.py" = ["E402"]` to `ruff.toml`. |
30      | **Ops / maintenance tool** (not part of the served app ? cache GC, batch admin) |
`tools/` | Run as `python -m tools.<name>` (it's an importable package). e.g.
`tools/cleanup_caches.py`. Keep the destructive decision path pure + unit-tested. |
31      | Training code (model fitting) | `training/` | Behind the CI-enforced import wall
(ADR-008) ? `backend.main` must not import it. Own `requirements-train.txt`. |
32      | A test | `tests/test_*.py` | pytest auto-discovers; the full-suite lane has a **70%
coverage floor**. Pure/offline + mocked (no GPU/WSL/network). |
33      | A doc (design, plan, living log) | `docs/` | Index it in `docs/README.md`. Only
**terminal-state** (DONE/REJECTED) work moves to `docs/archives/`. |
34      | A throwaway repro / debug script | `scratch/` | **Gitignored** + excluded from
ruff/pytest. Never imported by app code. |
35      | A model artifact (weights + manifest) | `artifacts/<name>/<ver>/` | ADR-008:
**manifest committed** (provenance), **weights gitignored**. |
36      | Eval baseline / leaderboard JSON | `eval_baselines/` | Tracked (survives a clean
checkout); the no-regression gate reads it. |
37      | Pipeline run outputs (reels, trajectories, DBs, frames) | `output/` | **Gitignored.**
Per-video isolation is planned in `PER_VIDEO_OUTPUT_LAYOUT.md`. |
38
39      **Canonical files that stay at the repo ROOT** (referenced by CI / packaging / docs ? do
not move):
40      `pyproject.toml` (PEP 621 packaging, hatchling; `indexer` entry point) .
`run_all_tests.py` (CI: `ci.yml`) .
41      `run_regression.py` (the no-regression gate CLI, cited in `EXPERIMENT_HARNESS.md`) .
`run_eval.py`
42      (grandfathered ? imported by `tests/test_eval_harness.py`). The old diagnostic
`setup.py` moved to
43      `scripts/doctor.py` (it shadowed the build system); invoke via `python`
```

```

scripts/doctor.py` or `indexer --setup`.
44     New standalone run-drivers go in **`scripts/`, not the root.
45
46     ---
47
48     ## 1. PIPELINE
49
50     ### 1A. Runtime: video file -> highlight reel
51     ```
52     typed config load (built-in defaults -> config.json overrides; absent/parse-fail -> all-
defaults, never fatal)
53                                     @backend/config/loader.py::load_config ->
@backend/config/models.py::AppConfig
54     -> video_id = MD5(whole file)          @backend/utils/hashing.py::compute_video_id
(single shared helper, 64KB chunks; imported by main + all run_*.py ? no more per-entrypoint
copies)
55     -> validate
@backend/pipeline/validators/base.py::registry.run_all_with_context(video_path, global_config)
56     FormatValidator -> ctx['ffprobe_meta'] -> ResolutionFPSValidator -> PTSContinuity
-> SceneCut -> Blur -> Relevance    (? ACTUAL registration/exec order ? see
@backend/pipeline/validators/__init__.py)
57     [validation FAIL -> persist status="failed", return early; report STILL emitted
in finally]
58     -> db.add_video(video_id, filepath, status, [r.dict()])
@backend/infrastructure/database.py
59     -> seg_name = req.segmenter or global_config.indexing.default_segmenter (fallback
'motion')
60     -> seg = segmenter_registry.get(seg_name)(db, global_config)    ?
@backend/pipeline/segmenters/base.py::segmenter_registry    (400 + available list if missing)
61     -> segments = seg.process_video(video_id, video_path, player_pool?, max_frames?)
62     [STOP POINT: segments-only ? predictions now in DB; eval/regression operate here
without stitching]
63     -> finally: emit_indexer_report(...)    @backend/reporting/__init__.py    (? ALWAYS runs
? even on validation failure / exception; NEVER raises; grafts response["reports"])
64     -> select segment ids -> [(start,end)] (+ stitching padding)
65     -> VideoCompiler.compile_highlights(filepath, time_pairs, out_name)
@backend/pipeline/stitcher/compiler.py
66     per-clip ffmpeg re-encode (libx264/superfast/crf22) -> concat -c copy ->
output/<name>.mp4
67     ```
68     Hybrid segmenter internal 4-phase (yolo_hybrid / tracknet_hybrid / wasb_hybrid ? same
shape):
69     ```

```

48. assistant (2026-06-16T02:26:16.972Z)

I now have a comprehensive understanding of the test suite, fixture conventions, and the right placement for new per-video regression fixtures. Let me compile my findings.

49. user (2026-06-16T02:27:43.513Z)

Structured output provided successfully

50. assistant (2026-06-16T02:27:45.559Z)

I've completed the analysis and returned my findings via StructuredOutput.